# ORBIT AI: INTELLIGENT CYBER THREAT ANALYSIS AND POLICY-GUIDED AUTOMATED RESPONSE

## Project-I Report

*Submitted by*


| | |
|---|---|
| **A. MOHAMED ISMAIL** | **(95192309027)** |
| **K. SIVA SUBRAMANIAN** | **(95192309048)** |
| **M. KAVIN KUMAR** | **(95192309016)** |
| **P. RAM KISHORE** | **(95192309042)** |


***in partial fulfillment for the award of the degree***
***of***
**BACHELOR OF TECHNOLOGY**
***in***
**ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**


## P.S.R. ENGINEERING COLLEGE, SIVAKASI

**(An Autonomous Institution, Affiliated to Anna University, Chennai)**


**ANNA UNIVERSITY : CHENNAI 600 025**

**OCTOBER 2026**

## BONAFIDE CERTIFICATE

Certified that this project report titled "**Orbit AI: Intelligent Cyber Threat Analysis and Policy-Guided Automated Response**" is the Bonafide work of "**A. Mohamed Ismail (95192309027), K. Siva Subramanian (95192309048), M. Kavin Kumar (95192309016), P. Ram Kishore (95192309042)**" who carried out the project work under my supervision.


**SIGNATURE**
**Mr. D. JEBA KINGSLEY M.E.,**
**SUPERVISOR**
**ASSISTANT PROFESSOR**
Department of AI & DS
P.S.R. Engineering College,
Sivakasi - 626140.

**SIGNATURE**
**Dr C. NATARAJAN M.E., Ph.D.,**
**HEAD OF THE DEPARTMENT**
**ASSOCIATE PROFESSOR**
Department of AI & DS
P.S.R. Engineering College,
Sivakasi - 626140.


**Submitted for the PROJECT-I Viva-Voce Examination to be held on** [date]

**INTERNAL EXAMINER**

**EXTERNAL EXAMINER**

## ACKNOWLEDGEMENT

We take this opportunity to thank all those who helped towards the successful completion of this project. At the very outset, we thank the Almighty for His profuse blessings showered on us. We thank our beloved parents whose encouragement and support helped us to complete our project successfully.

It is our greatest pleasure to express our sincere thanks to **Thiru R. SOLAISAMY**, Correspondent and Director, **Er. S. VIGNESWARI ARUNKUMAR B.Tech.,** P.S.R. Engineering College, Sivakasi, for providing the required facilities and suitable infrastructure to carry out this project.

It is our privilege to convey our sincere thanks to our respected and honourable Principal **Dr. J. S. SENTHIL KUMAAR, M.E., Ph.D.,** for giving us permission to do this project in our institution.

We wish to extend and express our sincere thanks to our adored Head of the Department, **Dr. C. NATARAJAN, M.E., Ph.D., Associate Professor** for his excellent guidance and motivation to complete our project proficiently.

We also wish to express our hearty thanks to our Project Guide, **Mr. D. JEBA KINGSLEY, M.E., Assistant Professor** for his excellent guidance and constant encouragement during this project work.

We also wish to express our sincere thanks to our Esteemed Project Coordinators **Dr. C. NATARAJAN, M.E., Ph.D., Associate Professor And Miss. G. SEENIAMMAL, M.E., Assistant Professor** for their continuous guidance and valuable suggestions, without whom this project would not have become a reality.

We are also bound to thank all the Teaching and Non-Teaching staff members of our **Artificial Intelligence and Data Science Department** who supported us with their wishes and prayers that brought success and completion to our project.

## ABSTRACT

Orbit AI is a cybersecurity demonstration prototype that analyses sequences of security-related events, classifies potential threat patterns, calculates risk and supports policy-guided incident response with analyst oversight. Each event sequence is summarised by five features – failed logins, successful logins, sensitive access, privilege activity and sequence length – and classified by a Random Forest model (100 trees, maximum depth 8) into one of five categories: Monitoring, Authentication Anomaly, Account Compromise, Data Collection and Privilege Escalation. The model runs in a Python FastAPI service and is trained on approximately 3,000 synthetic records generated by the training pipeline. Its confidence value is the maximum class probability and is not a calibrated probability of a real attack.

A deterministic risk-scoring engine combines base severity, model confidence and asset criticality, and repeated events may raise the score. A rule-based policy engine selects or controls response actions, and selected actions require approval or rejection by an analyst. The prototype also provides account restoration, a security operations dashboard and a simulated enterprise portal, implemented with React 19 and Vite, Node.js and Express 5, and an SQLite database accessed through Sequelize. Screenshots of a demonstration session show the workflow from portal events to an incident, a risk report, a response recommendation and a blocked account.

The report distinguishes deterministic automation from generative-AI functionality and describes the system strictly as a prototype for demonstration and further evaluation. No accuracy, precision, recall or other performance measures have been established, the ML engine was in a fallback state when the screenshots were taken, and several implementation limitations remain to be re-verified. The report therefore sets out a testing and evaluation plan, the limitations of the current work and the improvements required before any real-world use.

## TABLE OF CONTENTS


| CHAPTER NO. | TITLE | PAGE NO. |
|---|---|---|
| | **ABSTRACT** | **iv** |
| | **LIST OF TABLES** | **ix** |
| | **LIST OF FIGURES** | **xi** |
| | **LIST OF ABBREVIATIONS** | **xii** |
| **1** | **INTRODUCTION** | **1** |
| | 1.1 INTRODUCTION | 1 |
| | 1.2 BACKGROUND OF CYBERSECURITY THREAT DETECTION | 1 |
| | 1.3 PROBLEM STATEMENT | 2 |
| | 1.4 MOTIVATION FOR EVENT-BASED THREAT ANALYSIS | 2 |
| | 1.5 NEED FOR POLICY-GUIDED RESPONSE | 2 |
| | 1.6 IMPORTANCE OF HUMAN OVERSIGHT | 3 |
| | 1.7 PROJECT SCOPE | 3 |
| | 1.8 PROJECT OBJECTIVES | 3 |
| | 1.9 EXPECTED CONTRIBUTIONS | 4 |
| | 1.10 ORGANISATION OF THE REPORT | 4 |
| | 1.11 RELATED PAPER SUBMISSIONS | 5 |
| **2** | **LITERATURE REVIEW** | **6** |
| | 2.1 INTRODUCTION | 6 |
| | 2.2 ANOMALY-BASED INTRUSION DETECTION | 6 |
| | 2.3 CYBERATTACK PREDICTION AND FORECASTING | 6 |
| | 2.4 MITRE ATT&CK AND THREAT KNOWLEDGE FRAMEWORKS | 6 |
| | 2.5 SECURITY LOG CLUSTERING AND EVENT ANALYSIS | 7 |
| | 2.6 MACHINE-LEARNING-BASED INTRUSION DETECTION | 7 |
| | 2.7 ATTACK-GRAPH VISUALISATION AND ANALYST COMPREHENSION | 7 |
| | 2.8 EXPLAINABLE AI FOR CYBERSECURITY | 8 |
| | 2.9 INTRUSION DETECTION AND PREVENTION SYSTEMS | 8 |
| | 2.10 SECURITY ORCHESTRATION AND DECLARATIVE APIs | 8 |
| | 2.11 LIMITATIONS OF MACHINE LEARNING IN OPERATIONAL SECURITY ENVIRONMENTS | 9 |
| | 2.12 MACHINE-LEARNING LIBRARY USED | 9 |
| | 2.13 SUMMARY OF THE LITERATURE | 9 |
| | 2.14 RESEARCH AND IMPLEMENTATION GAP | 10 |
| **3** | **EXISTING SYSTEM AND PROPOSED SYSTEM** | **11** |
| | 3.1 EXISTING SYSTEM | 11 |

| | | |
|---|---|---|
| | 3.2 CHALLENGES OF FRAGMENTED ALERTS AND MANUAL ANALYSIS | 11 |
| | 3.3 LIMITATIONS OF FULLY AUTOMATED RESPONSE WITHOUT OVERSIGHT | 11 |
| | 3.4 PROPOSED SYSTEM: THE ORBIT AI WORKFLOW | 11 |
| | 3.5 KEY FEATURES | 12 |
| | 3.6 ADVANTAGES AND LIMITATIONS | 13 |
| | 3.7 PROJECT SCOPE AND BOUNDARIES | 13 |
| **4** | **SYSTEM REQUIREMENTS AND ARCHITECTURE** | **15** |
| | 4.1 INTRODUCTION | 15 |
| | 4.2 FUNCTIONAL REQUIREMENTS | 15 |
| | 4.3 NON-FUNCTIONAL REQUIREMENTS | 16 |
| | 4.4 HARDWARE REQUIREMENTS | 16 |
| | 4.5 SOFTWARE REQUIREMENTS AND TECHNOLOGY STACK | 17 |
| | 4.6 SYSTEM ARCHITECTURE | 17 |
| | 4.7 COMMUNICATION BETWEEN FRONT END, BACK END AND ML SERVICE | 18 |
| | 4.8 DATABASE ARCHITECTURE | 19 |
| | 4.9 DATA FLOW | 19 |
| | 4.10 TRUST BOUNDARIES AND SECURITY CONSIDERATIONS | 20 |
| | 4.11 VERIFICATION STATUS OF IMPLEMENTATION CLAIMS | 21 |
| | 4.12 THREAT CONSIDERATIONS FOR THE PROTOTYPE ITSELF | 22 |
| **5** | **METHODOLOGY AND IMPLEMENTATION** | **23** |
| | 5.1 OVERVIEW OF THE METHODOLOGY | 23 |
| | 5.2 STEP-BY-STEP WORKFLOW | 23 |
| | 5.3 INPUT FEATURES | 24 |
| | 5.4 SYNTHETIC TRAINING DATA | 24 |
| | 5.5 THE RANDOM FOREST ALGORITHM | 25 |
| | 5.6 THREAT CLASSIFICATION CATEGORIES | 27 |
| | 5.7 CONFIDENCE SCORE | 27 |
| | 5.8 RISK ASSESSMENT ENGINE | 27 |
| | 5.9 POLICY ENGINE | 28 |
| | 5.10 RESPONSIBILITIES OF EACH COMPONENT | 29 |
| | 5.11 ANALYST APPROVAL AND RESPONSE HANDLING | 29 |
| | 5.12 DETERMINISTIC AUTOMATION VERSUS GENERATIVE AI | 30 |
| | 5.13 LOGGING AND DASHBOARD UPDATE | 30 |
| | 5.14 SUITABILITY CONSIDERATIONS FOR THE DESIGN CHOICES | 30 |
| **6** | **SYSTEM MODULES AND USER INTERFACE** | **31** |

6.1 INTRODUCTION 31
6.2 SECURITY OPERATIONS DASHBOARD 31
6.3 SECURITY EVENTS MODULE 32
6.4 THREAT DETECTION MODULE 33
6.5 INCIDENT QUEUE MODULE 34
6.6 ATTACK PATHS MODULE 35
6.7 RISK ASSESSMENT MODULE 36
6.8 RESPONSE WORKFLOW AND ANALYST APPROVAL MODULE 37
6.9 ACCOUNT RESTORATION MODULE 38
6.10 SIMULATED ENTERPRISE PORTAL 39
6.11 AUDIT LOGGING 40
6.12 SUMMARY OF MODULE DEPENDENCIES AND KNOWN ISSUES 41
**7 TESTING AND EVALUATION 42**
7.1 INTRODUCTION 42
7.2 SOFTWARE FUNCTIONALITY TESTING AND MACHINE-LEARNING EVALUATION 42
7.3 TEST ENVIRONMENT 42
7.4 TEST CASES 43
7.5 EXISTING AUTOMATED TESTS 45
7.6 EVALUATION MEASURES FOR THE MACHINE-LEARNING MODEL 45
**8 RESULTS AND DISCUSSION 47**
8.1 BASIS OF THE RESULTS 47
8.2 WHAT THE PROTOTYPE IS DESIGNED TO DEMONSTRATE 47
8.3 BEHAVIOUR OBSERVED IN THE SCREENSHOTS 47
8.4 SEQUENCE OF EVENTS IN THE CAPTURED SESSION 48
8.5 IMPORTANT OBSERVATION: ML ENGINE IN FALLBACK STATE 49
8.6 WHAT CANNOT YET BE CONCLUDED 49
8.7 EXPERIMENTS REQUIRED 49
8.8 TEMPLATES FOR RECORDING MODEL RESULTS 50
8.9 DISCUSSION 51
8.10 THREATS TO VALIDITY 51
**9 LIMITATIONS AND FUTURE ENHANCEMENTS 52**
9.1 INTRODUCTION 52
9.2 LIMITATIONS IDENTIFIED IN PRELIMINARY REVIEW 52
9.3 ADDITIONAL ISSUES OBSERVED IN THE SCREENSHOTS 53
9.4 DISCUSSION OF THE LIMITATIONS 53

9.5 REQUIREMENTS FOR REAL-WORLD DEPLOYMENT 54
9.6 FUTURE ENHANCEMENTS 54
9.7 ETHICAL AND RESPONSIBLE-USE CONSIDERATIONS 55
**10 CONCLUSION 56**
**REFERENCES 57**
**APPENDIX 58**
APPENDIX A: PROJECT TEAM AND GUIDE DETAILS 58
APPENDIX B: SOURCE-CODE VERIFICATION CHECKLIST 58
APPENDIX C: ADDITIONAL FIGURES TO BE INSERTED 59
APPENDIX D: RISK-SCORING AND POLICY SUMMARY TEMPLATES 59
APPENDIX E: TEST-CASE TEMPLATE 60
APPENDIX G: PUBLICATION AND ORIGINALITY EVIDENCE 60

## LIST OF TABLES


| TABLE NO. | TITLE | PAGE No. |
|---|---|---|
| 1.1 | ORGANISATION OF THE REPORT | 5 |
| 2.1 | SUMMARY OF THE REVIEWED LITERATURE AND ITS RELEVANCE TO ORBIT AI | 10 |
| 3.1 | KEY FEATURES OF ORBIT AI AS A PROTOTYPE | 13 |
| 3.2 | COMPARISON OF CONVENTIONAL MONITORING AND THE ORBIT AI PROTOTYPE (QUALITATIVE) | 14 |
| 4.1 | FUNCTIONAL REQUIREMENTS | 15 |
| 4.2 | NON-FUNCTIONAL REQUIREMENTS AND THEIR CURRENT STATUS | 16 |
| 4.3 | HARDWARE USED FOR DEVELOPMENT AND DEMONSTRATION | 16 |
| 4.4 | SOFTWARE REQUIREMENTS AND TECHNOLOGY STACK | 17 |
| 4.5 | IMPLEMENTATION CLAIMS AND VERIFICATION STATUS | 22 |
| 4.6 | THREATS TO THE PROTOTYPE AND REQUIRED MITIGATIONS | 22 |
| 5.1 | DESCRIPTION OF THE ELEVEN WORKFLOW STEPS | 24 |
| 5.2 | INPUT FEATURES OF THE CLASSIFICATION MODEL | 24 |
| 5.3 | MODEL CONFIGURATION | 26 |
| 5.4 | THREAT CLASSIFICATION CATEGORIES | 27 |
| 5.5 | DIVISION OF RESPONSIBILITIES IN THE ORBIT AI WORKFLOW | 29 |
| 6.1 | NAVIGATION STRUCTURE OF THE ORBIT AI INTERFACE | 31 |
| 6.2 | SECURITY OPERATIONS DASHBOARD MODULE | 32 |
| 6.3 | SECURITY EVENTS MODULE | 33 |
| 6.4 | THREAT DETECTION MODULE | 34 |
| 6.5 | INCIDENT QUEUE MODULE | 35 |
| 6.6 | ATTACK PATHS MODULE | 36 |
| 6.7 | RISK ASSESSMENT MODULE | 37 |
| 6.8 | RESPONSE WORKFLOW AND ANALYST APPROVAL MODULE | 38 |
| 6.9 | ACCOUNT RESTORATION MODULE | 39 |
| 6.10 | SIMULATED ENTERPRISE PORTAL MODULE | 40 |
| 6.11 | AUDIT LOGGING MODULE | 41 |
| 6.12 | SUMMARY OF MODULES, DEPENDENCIES AND RELATED ISSUES | 41 |
| 7.1 | TEST ENVIRONMENT | 42 |
| 7.2 | TEST CASES FOR THE ML SERVICE AND RISK ENGINE | 43 |
| 7.3 | TEST CASES FOR THE POLICY ENGINE AND ANALYST WORKFLOW | 44 |
| 7.4 | TEST CASES FOR THE USER INTERFACE AND INTEGRATION | 45 |
| 7.5 | TEST CASES FOR SECURITY AND DEPLOYMENT | 45 |
| 7.6 | EVALUATION MEASURES REQUIRED FOR THE CLASSIFIER | 46 |

| | | |
|---|---|---|
| 8.1 | BEHAVIOUR OBSERVED IN THE SUPPLIED SCREENSHOTS | 48 |
| 8.2 | TIMESTAMPS VISIBLE IN THE CAPTURED SESSION (10/10/2026) | 49 |
| 8.3 | EXPERIMENTS REQUIRED TO OBTAIN SUPPORTABLE RESULTS | 50 |
| 8.4 | CONFUSION MATRIX TEMPLATE (ROWS: TRUE CLASS; COLUMNS: PREDICTED CLASS) | 50 |
| 8.5 | PER-CLASS PERFORMANCE TEMPLATE | 50 |
| 9.1 | KNOWN LIMITATIONS, IMPLICATIONS AND PROPOSED REMEDIATION | 53 |
| 9.2 | ADDITIONAL ISSUES OBSERVED IN THE SUPPLIED SCREENSHOTS | 53 |
| 9.3 | PROPOSED FUTURE ENHANCEMENTS (NOT IMPLEMENTED) | 55 |
| A.1 | PROJECT TEAM | 58 |
| A.2 | PROJECT GUIDE | 58 |
| B.1 | ITEMS TO RECORD FROM THE SOURCE CODE | 58 |
| D.1 | RISK-SCORING PARAMETERS (TO BE COMPLETED FROM SOURCE) | 59 |
| D.2 | POLICY RULES (TO BE COMPLETED FROM SOURCE) | 59 |
| E.1 | BLANK TEST-CASE RECORD | 60 |
| G.1 | CONFERENCE SUBMISSION RECORDS | 60 |
| G.2 | TURNITIN SIMILARITY SUMMARY FOR THE CONFERENCE PAPER | 62 |

## LIST OF FIGURES


| FIGURE NO. | TITLE | PAGE No. |
|---|---|---|
| 3.1 | OVERALL ORBIT AI PROCESSING WORKFLOW (PROTOTYPE) | 12 |
| 4.1 | OVERALL SYSTEM ARCHITECTURE OF THE ORBIT AI PROTOTYPE | 18 |
| 4.2 | USE-CASE VIEW OF THE ORBIT AI PROTOTYPE | 19 |
| 4.3 | LEVEL-1 DATA-FLOW DIAGRAM OF THE ORBIT AI PROTOTYPE | 20 |
| 4.4 | TRUST BOUNDARIES OF THE ORBIT AI PROTOTYPE | 21 |
| 5.1 | EVENT-PROCESSING WORKFLOW OF ORBIT AI | 23 |
| 5.2 | CONCEPTUAL VIEW OF THE RANDOM FOREST ENSEMBLE | 25 |
| 5.3 | MACHINE-LEARNING TRAINING AND INFERENCE PIPELINE | 26 |
| 5.4 | RISK-ASSESSMENT AND POLICY-EVALUATION WORKFLOW | 28 |
| 5.5 | ANALYST APPROVAL WORKFLOW | 29 |
| 6.1 | SECURITY OPERATIONS DASHBOARD (OVERVIEW SCREEN) | 32 |
| 6.2 | SECURITY EVENT TELEMETRY SCREEN | 33 |
| 6.3 | ML THREAT DETECTION PIPELINE SCREEN | 34 |
| 6.4 | ACTIVE INCIDENT QUEUE SCREEN | 35 |
| 6.5 | ATTACK PATHS SCREEN FOR INCIDENT ALT-8329 | 36 |
| 6.6 | RISK ANALYSIS SCREEN FOR INCIDENT ALT-8329 | 37 |
| 6.7 | AGENTIC RESPONSE WORKFLOW SCREEN (RECOMMENDATIONS) | 38 |
| 6.8 | ACCOUNTS AND RECOVERY SCREEN | 39 |
| 6.9 | SIMULATED ENTERPRISE PORTAL (ACME CORP PORTAL, MY FILES) | 40 |
| A.1 | ENTITY-RELATIONSHIP DIAGRAM OF THE ORBIT AI DATABASE (SCREENSHOT TO BE INSERTED) | 59 |
| A.2 | DEPLOYMENT ARCHITECTURE OF THE ORBIT AI PROTOTYPE (SCREENSHOT TO BE INSERTED) | 59 |
| G.1 | CMT ACKNOWLEDGEMENT FOR PAPER ID 255 (CYPHER2026) | 60 |
| G.2 | CMT ACKNOWLEDGEMENT FOR PAPER ID 5 (SPECTRUM2027) | 61 |
| G.3 | CMT ACKNOWLEDGEMENT FOR PAPER ID 202 (HACER2026) | 61 |
| G.4 | TURNITIN INTEGRITY OVERVIEW FOR THE CONFERENCE PAPER | 62 |

## LIST OF ABBREVIATIONS

| ABBREVIATION | EXPANSION |
|---|---|
| AI | ARTIFICIAL INTELLIGENCE |
| API | APPLICATION PROGRAMMING INTERFACE |
| ATT&CK | ADVERSARIAL TACTICS, TECHNIQUES AND COMMON KNOWLEDGE |
| CMT | CONFERENCE MANAGEMENT TOOLKIT |
| CPU | CENTRAL PROCESSING UNIT |
| DB | DATABASE |
| DFD | DATA FLOW DIAGRAM |
| DS | DATA SCIENCE |
| FN | FALSE NEGATIVE |
| FP | FALSE POSITIVE |
| GenAI | GENERATIVE ARTIFICIAL INTELLIGENCE |
| HTTP | HYPERTEXT TRANSFER PROTOCOL |
| IDPS | INTRUSION DETECTION AND PREVENTION SYSTEM |
| IDS | INTRUSION DETECTION SYSTEM |
| JSON | JAVASCRIPT OBJECT NOTATION |
| ML | MACHINE LEARNING |
| ORM | OBJECT-RELATIONAL MAPPER |
| RBAC | ROLE-BASED ACCESS CONTROL |
| REST | REPRESENTATIONAL STATE TRANSFER |
| RNN | RECURRENT NEURAL NETWORK |
| SHAP | SHAPLEY ADDITIVE EXPLANATIONS |
| SIEM | SECURITY INFORMATION AND EVENT MANAGEMENT |
| SOAR | SECURITY ORCHESTRATION, AUTOMATION AND RESPONSE |
| SOC | SECURITY OPERATIONS CENTER |
| SQL | STRUCTURED QUERY LANGUAGE |
| TN | TRUE NEGATIVE |
| TP | TRUE POSITIVE |
| UI | USER INTERFACE |

# CHAPTER 1

# INTRODUCTION

## 1.1 INTRODUCTION

Modern organisations depend on authentication services, enterprise portals, databases and internal applications, and every user interaction with these systems produces security-relevant events such as logins, failed logins, access to sensitive resources and changes in privilege. Security analysts must continually decide which sequences of events are routine and which require investigation or intervention. Because the volume of events is large and individual events are often ambiguous, this decision is difficult to make consistently by manual inspection alone.

Orbit AI is a cybersecurity demonstration prototype designed to address this decision problem at a small scale. It analyses sequences of security-related events, classifies the pattern into a threat category using a machine-learning model, calculates a risk score through a deterministic scoring engine, applies rule-based security policies to select or control a response action, and keeps a human analyst in the loop for selected response actions. The project demonstrates how these four elements – machine-learning classification, deterministic risk assessment, policy rules and analyst approval – can be combined in a single cybersecurity operations workflow.

The system is presented throughout this report as a prototype intended for demonstration and further evaluation. It is not described, and should not be read, as a production-ready enterprise security platform. Where a feature is only partly implemented, or where evidence of correct behaviour has not yet been collected, the report says so explicitly.

## 1.2 BACKGROUND OF CYBERSECURITY THREAT DETECTION

Threat detection has traditionally been performed by intrusion detection and prevention systems (IDPS), which monitor events in a host or network, identify possible incidents and, in the case of prevention technologies, attempt to stop them [8]. Detection methods are commonly grouped into signature-based approaches, which match known attack patterns, and anomaly-based approaches, which compare observed behaviour against a profile of normal activity. Anomaly-based approaches can in principle reveal previously unseen behaviour, but they bring challenges such as false alarms and the difficulty of defining normal behaviour [1].

Machine learning has been widely applied to intrusion detection. Deep-learning models such as recurrent neural networks have been reported to classify network traffic records on benchmark data [5]. At the same time, Sommer and Paxson argue that detecting intrusions with machine learning is intrinsically harder than many other machine-learning applications, because of the high cost of errors, the diversity of real traffic and the difficulty of evaluation [11]. This tension between promising laboratory results and the demands of operational environments motivates a cautious, evidence-based approach to any machine-learning component in a security workflow.

Knowledge frameworks such as MITRE ATT&CK describe adversary behaviour as tactics and techniques and provide a common vocabulary for security teams [3]. Research on attack projection, prediction and forecasting examines how observed events can be used to anticipate later stages of an attack [2], and research on security orchestration examines how security tools and incident-response activities can be coordinated through structured automation [9], [10]. Orbit AI draws on these research directions in a deliberately simplified form.

### 1.3 PROBLEM STATEMENT

Security alerts are frequently generated by separate components and presented as isolated records. An analyst who sees one failed login has little reason for concern, whereas the same analyst viewing a burst of failed logins followed by a successful login, access to sensitive resources and a privilege change would reach a very different conclusion. Manually correlating such events across many alerts is slow and error-prone. Conversely, fully automated responses that act without review can disrupt legitimate users or business operations when the underlying classification is wrong.

The problem addressed by this project is therefore to demonstrate a workflow in which (i) a sequence of related security events is summarised as a small set of features, (ii) a machine-learning model assigns the sequence to a threat category, (iii) a deterministic engine converts the category, the model confidence and contextual factors into a risk score, (iv) rule-based policies decide how the system should respond, and (v) selected response actions are held for explicit approval or rejection by a human analyst, with the outcome recorded and displayed.

### 1.4 MOTIVATION FOR EVENT-BASED THREAT ANALYSIS

Many attack behaviours are visible only as patterns across events rather than in any single event. Credential guessing appears as repeated failed logins; account compromise may appear as failures followed by a success; data collection may appear as repeated access to sensitive resources; and privilege escalation may appear as unusual privilege-related activity. Representing a sequence of events by features such as the number of failed logins, successful logins, sensitive accesses, privilege activities and the sequence length allows a classifier to learn how these quantities combine.

Event-based analysis is also compatible with the way security knowledge is organised in practice. Frameworks such as ATT&CK describe adversary behaviour in terms of observable techniques [3], and surveys of log analysis emphasise the value of grouping and interpreting large volumes of heterogeneous log events [4]. Orbit AI does not implement a formal mapping to ATT&CK or a log-clustering method; it uses event-based features as a compact and transparent representation suitable for a demonstration prototype.

### 1.5 NEED FOR POLICY-GUIDED RESPONSE

A classification result by itself does not tell an organisation what to do. Responses must follow organisational rules about which actions are permitted, which require authorisation and how the severity of a situation should be treated. Security orchestration research highlights the importance

of structured automation in coordinating cybersecurity operations [9], [10]. Orbit AI addresses a very small part of this space by expressing response decisions as deterministic rules, so that the same inputs always lead to the same recommended action and the reasoning can be inspected and tested.

### 1.6 IMPORTANCE OF HUMAN OVERSIGHT

Automated responses must be carefully controlled to reduce the risk of disruptive or inappropriate actions. A model trained on limited or synthetic data may misclassify a legitimate sequence, and an action such as restricting an account can interrupt genuine work. Orbit AI therefore supports analyst approval for selected response actions before they proceed, and provides a way to restore an affected account. This human-in-the-loop approach is intended to balance response efficiency with operational oversight. The current implementation demonstrates the workflow in a prototype environment; its effectiveness in production settings requires further evaluation.

### 1.7 PROJECT SCOPE

The scope of the project covers the design and implementation of a working prototype consisting of a web-based security operations interface, a backend application with a database, and a separate machine-learning service. Within this scope, the prototype:

- accepts or generates simulated security events, including events originating from a simulated enterprise portal;
- classifies event patterns into five threat categories using a Random Forest model;
- calculates a deterministic risk score and applies rule-based policies to select response actions;
- supports analyst approval or rejection of selected actions and account restoration;
- records incident-related information, including audit records, and displays it on a dashboard.

The following are outside the scope of the project: integration with real enterprise security-event sources, evaluation on real-world benchmark datasets, production-grade authentication and authorisation, and deployment in a live environment. The model is trained on synthetic data produced by the project's own training pipeline, so the prototype makes no claim of validated real-world detection performance.

### 1.8 PROJECT OBJECTIVES

1. To study existing research on anomaly-based intrusion detection, attack prediction, threat knowledge frameworks, explainable AI and security orchestration, and to identify the gap that a prototype can usefully demonstrate.
2. To design an event-based representation using five features: failed logins, successful logins, sensitive access, privilege activity and sequence length.
3. To implement a machine-learning microservice that classifies event patterns into five threat categories and returns a model confidence value.

4. To implement a deterministic risk-scoring engine and a rule-based policy engine that select or control response actions.
5. To implement an analyst workflow for approving or rejecting selected response actions and for restoring accounts.
6. To provide a dashboard, audit records and a simulated enterprise portal that make the workflow observable.
7. To document honestly the limitations of the prototype and the work required before any real-world deployment.

## 1.9 EXPECTED CONTRIBUTIONS

The contribution of the project is a transparent, end-to-end demonstration rather than a performance advance. Specifically, it contributes (i) a clear separation between the responsibilities of the machine-learning model, the deterministic risk engine, the policy engine and the human analyst; (ii) a working integration of a React front end, a Node.js and Express backend with an SQLite database, and a Python FastAPI machine-learning service; (iii) a documented set of limitations and a verification plan that can guide further evaluation; and (iv) a baseline implementation that can be extended with real data and stronger security controls.

## 1.10 ORGANISATION OF THE REPORT

The remainder of the report is organised as summarised in Table 1.1.

| Chapter | Content |
|---|---|
| 1 | Introduction: background, problem statement, motivation, scope, objectives and contributions. |
| 2 | Literature review: anomaly-based detection, attack prediction, ATT&CK, log analysis, machine-learning detection, explainability, IDPS, security orchestration and the research gap. |
| 3 | Existing and proposed system: conventional monitoring, challenges, the Orbit AI workflow, features, advantages, limitations and boundaries. |
| 4 | System requirements and architecture: functional and non-functional requirements, technology stack, architecture, data flow and trust boundaries. |
| 5 | Methodology and implementation: the eleven-step workflow, Random Forest model, risk engine, policy engine and analyst approval. |
| 6 | System modules and user interface: purpose, input, processing and output of each module. |
| 7 | Testing and evaluation: test-case design, evidence status and the evaluation measures required for the model. |
| 8 | Results and discussion: what the prototype demonstrates and what cannot yet be concluded. |
| 9 | Limitations and future enhancements. |

| Chapter | Content |
|---|---|
| 10 | Conclusion. |

**Table 1.1 Organisation of the Report**

## 1.11 RELATED PAPER SUBMISSIONS

A conference paper with the same title as this report has been submitted to three conferences through the Microsoft Conference Management Toolkit (CMT). The submission acknowledgements confirm only that the submissions were created; no review outcome is available at the time of writing. The acknowledgements and the similarity report for the paper are reproduced in Appendix G. The technical description in the paper differs in some respects from the description in this report, and the report follows the project summary and the supplied screenshots.

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 INTRODUCTION

This chapter reviews the research areas that are relevant to Orbit AI: anomaly-based intrusion detection, cyberattack prediction, threat knowledge frameworks, security-log analysis, machine-learning-based intrusion detection, attack-graph visualisation, explainable AI, intrusion detection and prevention systems, security orchestration and the practical limits of machine learning in operational environments. The cited works are used to place the project in context. They do not validate the Orbit AI implementation, and no result from these papers is claimed as a result of this project.

## 2.2 ANOMALY-BASED INTRUSION DETECTION

García-Teodoro et al. survey anomaly-based network intrusion detection, organising the techniques that have been proposed, the systems that implement them and the challenges that remain [1]. The central idea of the anomaly-based approach is that behaviour deviating from a model of normal activity may indicate an intrusion, which allows previously unseen attacks to be flagged. The survey also discusses practical difficulties, including the need to obtain representative data to model normal behaviour and the tendency of such systems to raise false alarms.

Relevance to Orbit AI: the prototype does not build a profile of normal behaviour. It uses a supervised classifier trained to assign one of five labels to a feature vector, one of which (Monitoring) represents activity that needs no escalation. The class named Authentication Anomaly is a label in this classifier and should not be confused with unsupervised anomaly detection. The survey's discussion of false alarms and data requirements is nevertheless directly relevant to the evaluation needs described in Chapter 7.

## 2.3 CYBERATTACK PREDICTION AND FORECASTING

Husák et al. present a survey of attack projection, prediction and forecasting in cyber security, reviewing methods that use observed security data to anticipate how an attack may progress or what may occur next [2]. Such work motivates the view that events should be interpreted in sequence and in context, rather than in isolation.

Relevance to Orbit AI: Orbit AI classifies the pattern represented by a sequence of events that has already occurred. It does not forecast future attacker actions or project attack paths, and no claim of predictive capability is made.

## 2.4 MITRE ATT&CK AND THREAT KNOWLEDGE FRAMEWORKS

Strom et al. describe the design and philosophy of MITRE ATT&CK, a knowledge base that organises adversary behaviour, based on observed real-world activity, into tactics and techniques [3].

Its value for security operations lies in giving analysts, tools and researchers a shared vocabulary for describing and comparing behaviour.

Relevance to Orbit AI: some of the five Orbit AI categories, such as Data Collection and Privilege Escalation, resemble the names of ATT&CK tactics, and the categories are chosen for the same reason, namely to describe behaviour in an operationally meaningful way. However, the project does not implement a formal, validated mapping to ATT&CK tactics or techniques. Such a mapping is proposed as future work in Chapter 9.

## 2.5 SECURITY LOG CLUSTERING AND EVENT ANALYSIS

Landauer et al. survey system-log clustering approaches for cyber security applications, covering how large collections of log lines can be grouped to support tasks such as anomaly detection and event analysis, and discussing the challenges that arise from the volume and heterogeneity of log data [4].

Relevance to Orbit AI: the survey emphasises that raw log events must be abstracted before they can be analysed effectively. Orbit AI performs a much simpler abstraction by summarising a sequence of events into five numerical features. It does not cluster logs or parse raw log text.

## 2.6 MACHINE-LEARNING-BASED INTRUSION DETECTION

Yin et al. propose an intrusion detection approach based on recurrent neural networks and report its evaluation on the NSL-KDD benchmark dataset for both binary and multi-class classification [5]. Their work illustrates how learned models can be applied to labelled intrusion records and how multi-class classification of attack categories is framed.

Relevance to Orbit AI: Orbit AI also performs multi-class classification, but with a Random Forest rather than a recurrent network, on five hand-designed features rather than raw traffic records, and on synthetic data rather than a public benchmark. The results reported in [5] therefore cannot be transferred to Orbit AI, and evaluating the prototype on suitable public datasets is listed as future work.

## 2.7 ATTACK-GRAPH VISUALISATION AND ANALYST COMPREHENSION

Noel and Jajodia address the problem that attack graphs quickly become too large to understand, and propose managing this complexity through visual hierarchical aggregation [6]. The underlying point is that security information must be summarised in a form that an analyst can absorb.

Relevance to Orbit AI: the project does not implement attack graphs. The reference is relevant to the design of the dashboard, whose purpose is to summarise incident information so that an analyst can review it quickly.

## 2.8 EXPLAINABLE AI FOR CYBERSECURITY

Charmet et al. provide a literature survey of explainable artificial intelligence for cybersecurity [7]. A recurring theme in this area is that analysts need to understand why a model produced a given output before they can trust or act on it.

Relevance to Orbit AI: the prototype returns a threat category and a confidence value and displays them to the analyst, together with the risk score and recommendation. It does not currently implement a feature-attribution or other explainability method. Techniques such as SHAP are therefore listed in Chapter 9 as possible future work, not as completed features.

## 2.9 INTRUSION DETECTION AND PREVENTION SYSTEMS

The NIST guide to intrusion detection and prevention systems describes the types of IDPS technologies, the detection methodologies they use and the capabilities and limitations of each, and gives recommendations for their deployment and management [8]. It is a standard reference for the terminology of detection and prevention.

Relevance to Orbit AI: an IDPS observes real systems and may block activity automatically. Orbit AI, in contrast, operates on simulated events within a demonstration environment, and its response actions are subject to policy rules and, for selected actions, analyst approval. It should not be treated as an IDPS.

## 2.10 SECURITY ORCHESTRATION AND DECLARATIVE APIs

Islam, Babar and Nepal present a multi-vocal review of security orchestration, drawing on academic and practitioner sources to describe what security orchestration is, what functionalities it provides and what benefits and challenges it brings [9]. In a later article they introduce a declarative-API approach to security orchestration, called DecOr, intended to ease the adaptation of incident-response processes and security tools in orchestration platforms, and they demonstrate it by executing 48 incident-response processes with seven security tools [10].

Security orchestration research highlights the importance of structured automation in coordinating cybersecurity operations [9], [10]. However, automated responses must be carefully controlled to reduce the risk of disruptive or inappropriate actions. Orbit AI addresses this concern through a policy-guided response workflow that supports analyst approval for selected actions before execution. This human-in-the-loop approach is intended to balance response efficiency with operational oversight. The current implementation demonstrates this workflow in a prototype environment; its effectiveness in production settings requires further evaluation.

Relevance to Orbit AI: Orbit AI’s policy engine is a small set of deterministic rules and does not implement declarative APIs, ontologies or integration with external security tools. The cited work shows what a mature orchestration approach involves and thus indicates the direction in which the prototype would have to be developed.

## 2.11 LIMITATIONS OF MACHINE LEARNING IN OPERATIONAL SECURITY ENVIRONMENTS

Sommer and Paxson examine why machine learning has seen limited operational adoption for network intrusion detection despite a large research literature [11]. They argue that detecting novel attacks is fundamentally different from the pattern-recognition tasks in which machine learning is most successful, and they discuss the high cost of errors, the semantic gap between a detected anomaly and an actionable conclusion, the great variability of real network traffic and the difficulty of evaluating detectors convincingly.

Relevance to Orbit AI: this paper is the main reason the project is positioned as a demonstration prototype. Orbit AI is trained on synthetic data, has not been evaluated on an independent labelled test set, and has not been tested on real enterprise traffic. Consistent with [11], the report avoids claims of reliable real-world detection and emphasises analyst oversight for consequential actions.

## 2.12 MACHINE-LEARNING LIBRARY USED

Pedregosa et al. describe Scikit-learn, a Python library that provides a consistent interface to many machine-learning algorithms [12]. The Orbit AI machine-learning service uses the RandomForestClassifier implementation from this library. The reference supports the choice of tooling; it does not provide evidence about the accuracy of the Orbit AI model.

## 2.13 SUMMARY OF THE LITERATURE

| Ref. | Area | Focus of the work | Relevance to Orbit AI (no validation implied) |
|---|---|---|---|
| [1] | Anomaly-based IDS | Survey of techniques, systems and challenges | Context for false alarms and data needs; Orbit AI is supervised classification |
| [2] | Attack prediction | Survey of projection, prediction and forecasting | Supports sequence-aware analysis; Orbit AI does not forecast |
| [3] | Threat knowledge | Design and philosophy of MITRE ATT&CK | Context for category naming; no formal mapping implemented |
| [4] | Log analysis | Survey of log clustering for security | Motivates event abstraction; Orbit AI uses five features |
| [5] | ML-based IDS | RNN-based intrusion detection (NSL-KDD) | Example of multi-class detection; results not transferable |
| [6] | Visualisation | Attack-graph complexity management | Motivates concise dashboards; no attack graphs implemented |
| [7] | Explainable AI | Survey of XAI for cybersecurity | Highlights explainability need; not yet implemented |
| [8] | IDPS | NIST guide to IDPS | Terminology; Orbit AI is not an IDPS |

| Ref. | Area | Focus of the work | Relevance to Orbit AI (no validation implied) |
|---|---|---|---|
| [9] | Orchestration | Multi-vocal review of security orchestration | Context for structured, controlled automation |
| [10] | Orchestration APIs | Declarative APIs for orchestration (DecOr) | Indicates mature direction; not implemented in Orbit AI |
| [11] | ML limitations | Why ML is hard to deploy for intrusion detection | Basis for cautious positioning and evaluation plan |
| [12] | ML library | Scikit-learn library | Tooling for the Random Forest model |

**Table 2.1 Summary of the Reviewed Literature and Its Relevance to Orbit AI**

## 2.14 RESEARCH AND IMPLEMENTATION GAP

The reviewed literature treats detection, explanation and orchestration largely as separate topics, and it consistently cautions that machine-learning results do not automatically carry over to operational settings [1], [11]. Few small-scale, openly inspectable demonstrations exist that show, in one compact workflow, how a machine-learning classification, a deterministic risk score, explicit policy rules and a human approval step relate to one another.

Orbit AI attempts to demonstrate this combination as a prototype. The gap it targets is a demonstration gap rather than a performance gap: it shows how responsibilities can be separated and how analyst oversight can be placed in the response path. It does not attempt to show improved detection accuracy, reduced response time or suitability for production deployment, and the evidence required for such claims is described in Chapters 7 and 8.

# CHAPTER 3

# EXISTING SYSTEM AND PROPOSED SYSTEM

## 3.1 EXISTING SYSTEM

In conventional security operations, monitoring is performed by combinations of intrusion detection and prevention systems, log collection tools and rule-based alerting [8]. Alerts are usually raised when an individual condition is met, for example when a threshold of failed logins is crossed or a known signature is observed. Analysts then triage the alerts, investigate related records and decide on a response, often using separate tools for each step.

Orchestration platforms attempt to reduce the manual effort by coordinating tools and automating incident-response activities [9], [10]. These platforms are far more capable than Orbit AI in terms of integration and scale. They are mentioned here only to describe the existing landscape; the project makes no comparison of effectiveness with them.

## 3.2 CHALLENGES OF FRAGMENTED ALERTS AND MANUAL ANALYSIS

- **Fragmented alerts.** Related events can appear as separate alerts, so the pattern across them is not immediately visible.
- **Manual correlation.** Analysts must combine information from several views before judging whether a sequence is a threat, which is slow and depends on individual experience.
- **Inconsistent prioritisation.** Without a defined scoring method, two analysts may rank the same situation differently.
- **Weak linkage between detection and response.** The decision of what to do next is frequently not tied to a documented rule, which makes it difficult to review or test.
- **Traceability.** Reconstructing who decided what, and why, requires complete records of detections and decisions.

## 3.3 LIMITATIONS OF FULLY AUTOMATED RESPONSE WITHOUT OVERSIGHT

Automating the entire response chain reduces delay but introduces its own risks. A misclassification can lead directly to an inappropriate action such as restricting a legitimate user, and the cost of such errors in intrusion detection is high [11]. Automated actions also need to be governed by explicit rules and accompanied by records so that they can be audited. For these reasons Orbit AI places a human approval step in the path of selected response actions.

## 3.4 PROPOSED SYSTEM: THE ORBIT AI WORKFLOW

Orbit AI processes a sequence of security-related events as follows. The events are summarised into five features and sent to a machine-learning service, which returns a threat category and a model confidence value. A deterministic risk-scoring engine combines the category-dependent

base severity, the model confidence, the asset criticality and the effect of repeated events into a risk score. A rule-based policy engine uses the result to select or control a response action. Where a selected action requires approval, it is held until an analyst approves or rejects it. The outcome is recorded and shown on the security operations dashboard. Figure 3.1 summarises the workflow.

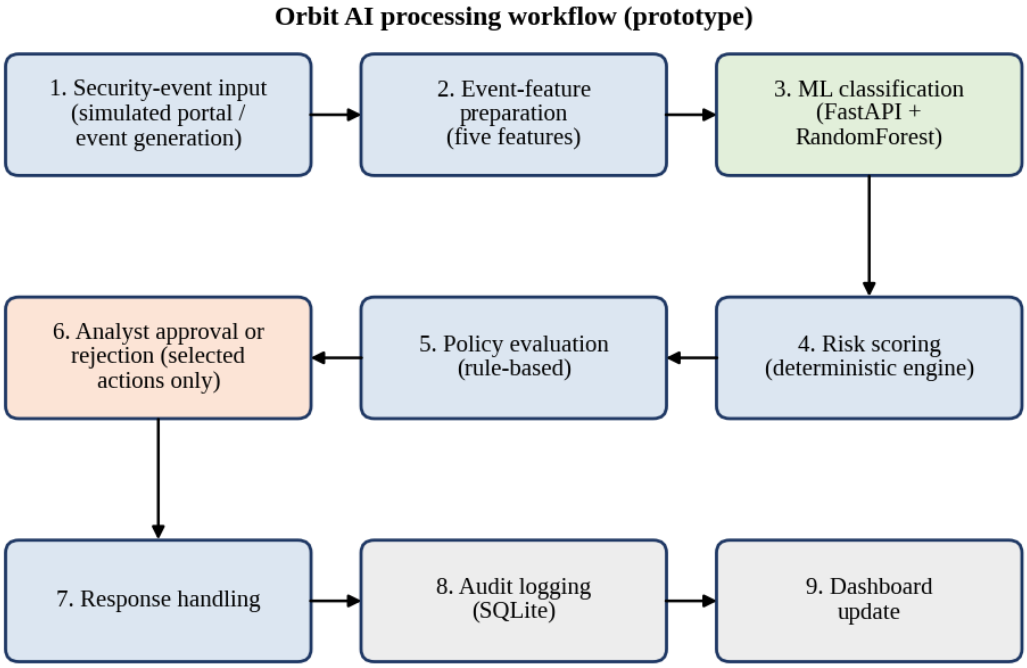



**Figure 3.1 Overall Orbit AI Processing Workflow (Prototype)**

The workflow makes a clear distinction between components that learn from data (the machine-learning classifier) and components that follow explicitly written logic (the risk-scoring and policy engines), and between automated steps and steps that require a human decision. This distinction is developed in detail in Chapter 5.

## 3.5 KEY FEATURES

| Feature | Description | Nature |
|---|---|---|
| Event-based classification | Five-feature representation of an event sequence classified into five threat categories by a Random Forest | Machine learning (synthetic training data) |
| Model confidence | Maximum class probability returned by predict_proba for the predicted class | Model output |
| Risk scoring | Score derived from base severity, confidence, asset criticality and repeated events | Deterministic engine |
| Policy-guided response | Rules select or control response actions | Rule-based logic |
| Analyst approval | Approval or rejection of selected response actions | Human decision |
| Account restoration | Analyst-initiated restoration of an affected account | Analyst workflow |

| Feature | Description | Nature |
|---|---|---|
| Audit logs | Records of workflow activity (coverage to be verified) | Persistence |
| Security operations dashboard | Display of incident-related information | User interface |
| Simulated enterprise portal | Demonstration environment from which activity can be simulated | Demonstration |

**Table 3.1 Key Features of Orbit AI as a Prototype**

## 3.6 ADVANTAGES AND LIMITATIONS

### 3.6.1 Advantages of the approach

- Responsibilities are separated, so each component can be tested and improved independently.
- The risk and policy engines are deterministic, so identical inputs produce identical outputs, which supports repeatable testing.
- Analyst approval for selected actions limits the effect of a wrong classification.
- The architecture uses common, open technologies and a separate ML service, which makes it straightforward to replace or retrain the model.

### 3.6.2 Limitations of the prototype

- The model is trained on synthetic data and has no demonstrated real-world accuracy.
- No accuracy, precision, recall, F1-score or benchmark results are available.
- Asset criticality is set to HIGH for incoming alerts, so it is not a dynamic asset-management feature.
- Several implementation issues identified in a preliminary code review (Chapter 9) need to be re-verified and resolved.
- No claim is made that Orbit AI outperforms existing systems.

## 3.7 PROJECT SCOPE AND BOUNDARIES

| Aspect | Conventional monitoring and orchestration | Orbit AI prototype |
|---|---|---|
| Data source | Real network, host and application telemetry | Simulated or generated events |
| Detection basis | Signatures, rules and anomaly profiles [8], [1] | Random Forest on five features, trained on synthetic data |
| Prioritisation | Tool-specific scoring | Deterministic risk-scoring engine |
| Response | Manual or orchestrated [9], [10] | Rule-based policy with analyst approval for selected actions |

| Aspect | Conventional monitoring and orchestration | Orbit AI prototype |
|---|---|---|
| Maturity | Operational products | Demonstration prototype; not production-ready |
| Performance evidence | Product-specific | None yet; to be established (Chapter 7) |

**Table 3.2 Comparison of Conventional Monitoring and the Orbit AI Prototype (Qualitative)**

Table 3.2 is intended only to describe differences in design. It does not rank the systems, and no performance advantage of Orbit AI is asserted. The boundaries of the project are the ones stated in Section 1.7: simulated data, a single-organisation demonstration setting and the absence of production-grade security controls.

# CHAPTER 4

# SYSTEM REQUIREMENTS AND ARCHITECTURE

## 4.1 INTRODUCTION

This chapter specifies the requirements of the Orbit AI prototype and describes its architecture. The technology descriptions are based on the project's technical summary. Section 4.10 lists each claim together with its verification status, because the source code must be inspected to confirm every item before final submission.

## 4.2 FUNCTIONAL REQUIREMENTS

| ID | Requirement | Module |
|---|---|---|
| FR1 | The system shall accept or generate security-related events for analysis. | Event intake |
| FR2 | The system shall derive five features from an event sequence: failed logins, successful logins, sensitive access, privilege activity and sequence length. | Feature preparation |
| FR3 | The system shall classify the sequence into one of five categories: Monitoring, Authentication Anomaly, Account Compromise, Data Collection or Privilege Escalation. | ML service |
| FR4 | The system shall return a model confidence value derived from the maximum class probability. | ML service |
| FR5 | The system shall calculate a deterministic risk score using base severity, model confidence and asset criticality, and shall allow repeated events to increase the score. | Risk engine |
| FR6 | The system shall evaluate rule-based policies to select or control a response action. | Policy engine |
| FR7 | The system shall hold selected response actions for analyst approval or rejection. | Response workflow |
| FR8 | The system shall allow an analyst to restore an affected account. | Account restoration |
| FR9 | The system shall record audit information about workflow activity. | Audit logging |
| FR10 | The system shall present incident-related information on a security operations dashboard. | Dashboard |
| FR11 | The system shall provide a simulated enterprise portal for demonstration. | Portal |

**Table 4.1 Functional Requirements**

## 4.3 NON-FUNCTIONAL REQUIREMENTS

| Attribute | Requirement | Status in the prototype |
|---|---|---|
| Separation of concerns | ML inference, risk, policy and presentation should be separable components. | Intended by the architecture; to be confirmed |
| Determinism | Risk and policy decisions should be repeatable for identical inputs. | Intended; to be confirmed by tests |
| Auditability | Detections and decisions should be recorded completely. | Audit coverage may be incomplete (Chapter 9) |
| Security | Access to actions should depend on verified identity and role. | A role request header is used; not robust (Chapter 9) |
| Data persistence | Stored data should survive restarts. | Start-up database behaviour needs review (Chapter 9) |
| Usability | The interface should present status and decisions clearly to an analyst. | Demonstrated through the dashboard; to be evaluated |
| Maintainability | Components should be replaceable (for example the model). | ML service is separate; to be confirmed |
| Performance | Inference latency should be low enough for interactive use. | Not measured |

**Table 4.2 Non-Functional Requirements and Their Current Status**

## 4.4 HARDWARE REQUIREMENTS

The prototype is a software demonstration and the machine-learning component is a Random Forest executed by Scikit-learn on a central processor, so no special accelerator hardware is expected to be necessary. The hardware actually used for development and demonstration is recorded in Table 4.3.

| Item | Specification |
|---|---|
| Processor | [to be completed: processor model] |
| Memory | [to be completed: installed RAM] |
| Storage | [to be completed: free disk space] |
| Display | [to be completed] |
| Network | [to be completed: local network used for the three services] |

**Table 4.3 Hardware Used for Development and Demonstration**

## 4.5 SOFTWARE REQUIREMENTS AND TECHNOLOGY STACK

| Layer | Technology | Role |
|---|---|---|
| Front end | React 19, Vite | Security operations dashboard and related interface modules |
| Back end | Node.js, Express 5 | Application programming interface, risk and workflow logic |
| Database | SQLite with Sequelize ORM | Persistence of application data |
| ML service | Python, FastAPI, Scikit-learn [12] | Hosts the RandomForestClassifier and serves predictions |
| Model | RandomForestClassifier: 100 trees, maximum depth 8 | Five-class threat classification |
| Other dependency | Google GenAI package (listed as a dependency) | Not confirmed as operational (Section 5.12) |
| Operating system | [to be completed] | Development and demonstration platform |
| Runtime versions | [Node.js and Python versions to be completed from the project files] | Execution environment |

**Table 4.4 Software Requirements and Technology Stack**

## 4.6 SYSTEM ARCHITECTURE

Orbit AI is organised into four tiers, shown in Figure 4.1. The React front end provides the interface. The Node.js and Express back end exposes the application programming interface used by the front end, hosts the application logic for events, risk scoring, policy evaluation, approvals, account restoration and audit logging, and stores data in an SQLite database through the Sequelize object-relational mapper. A separate Python service built with FastAPI hosts the Random Forest model and returns a threat category and a confidence value.

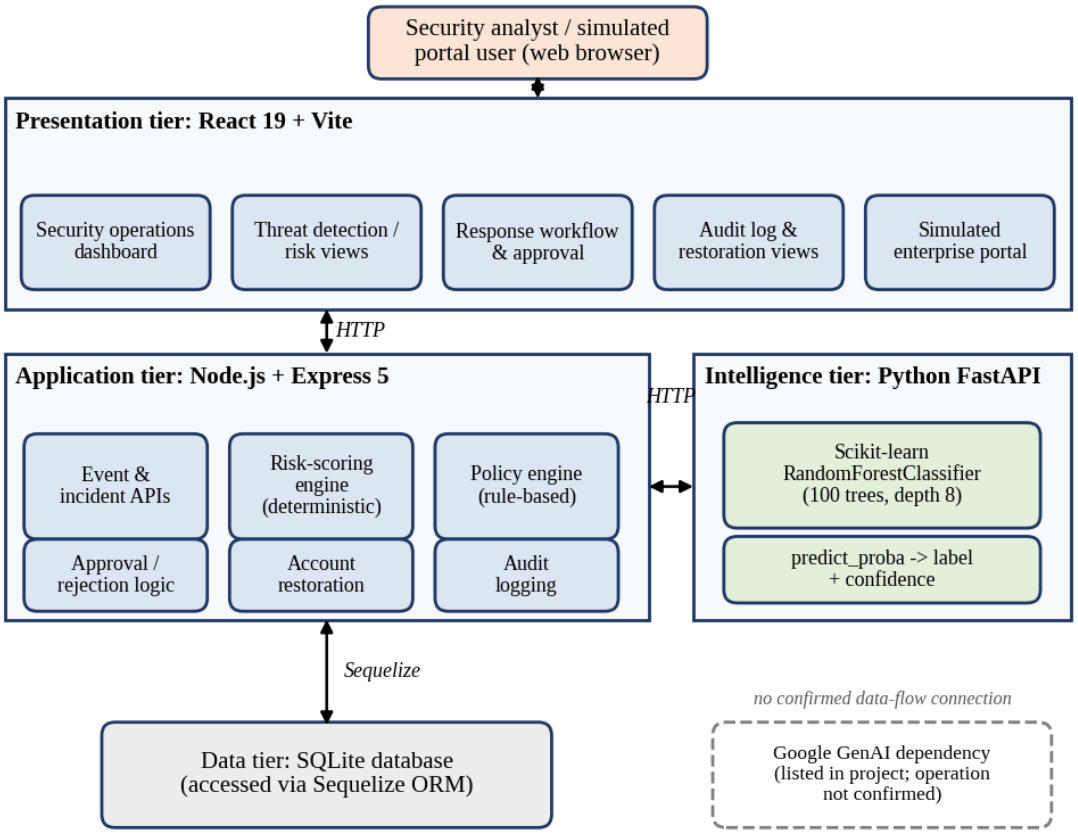



**Figure 4.1 Overall System Architecture of the Orbit AI Prototype**

The figure shows only those connections that are described in the project summary. The Google GenAI package is drawn as an unconnected, dashed element because the project summary lists it as a dependency but does not confirm that it is used in any operational data flow. The placement of the risk-scoring and policy modules inside the back end reflects the project description and is to be confirmed against the source code.

## 4.7 COMMUNICATION BETWEEN FRONT END, BACK END AND ML SERVICE

The front end communicates with the back end over HTTP. The back end communicates with the ML service over HTTP to obtain a prediction for the feature vector of an event sequence. The ML service returns the predicted category and the class probabilities from which the confidence is derived. The back end then computes the risk score and evaluates policy, stores the results and makes them available to the front end. In a containerised deployment the back end must address the ML service by its service name on the container network rather than by localhost; a preliminary review suggested that the Docker configuration may not do this (Chapter 9).

The exact endpoint names, request formats and response formats are implementation details to be recorded in Appendix B from the source code.

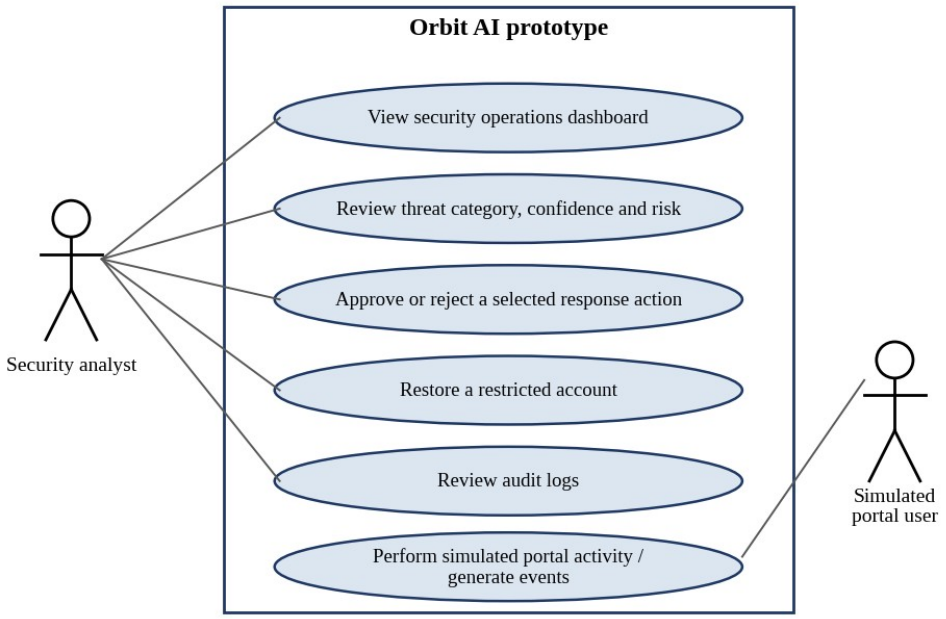
## 4.8 DATABASE ARCHITECTURE

The prototype stores application data in an SQLite database accessed through Sequelize. SQLite is a lightweight file-based database that is convenient for a demonstration prototype but is not designed for concurrent, multi-user enterprise workloads. The data that the workflow needs to persist includes incident-related records, response and approval states, account status and audit records. The precise tables and relationships are defined by the Sequelize models and are to be documented from the source code; an entity-relationship diagram will be added as Figure A.1 in Appendix C once the schema has been verified, and it is not drawn here because the schema has not been supplied.

A preliminary review indicated that the back end may call the Sequelize synchronisation function with the force option enabled, which drops and recreates tables each time the server starts. If confirmed, this would erase stored data at every start-up. This is treated as an open limitation in Chapter 9.

## 4.9 DATA FLOW

Figure 4.2 shows the use cases that the prototype is described as supporting, and Figure 4.3 shows the principal data flows. An event enters through the simulated portal or an event generator and is converted to features. The features are sent to the ML service, and the returned label and confidence are passed to the risk engine. The risk score is then passed to the policy engine, whose decision either proceeds under policy or is held for analyst approval. Results are written to the database and read by the dashboard and audit-log views.




*Links inferred from the described features; to be confirmed against the source code.*

**Figure 4.2 Use-Case View of the Orbit AI Prototype**

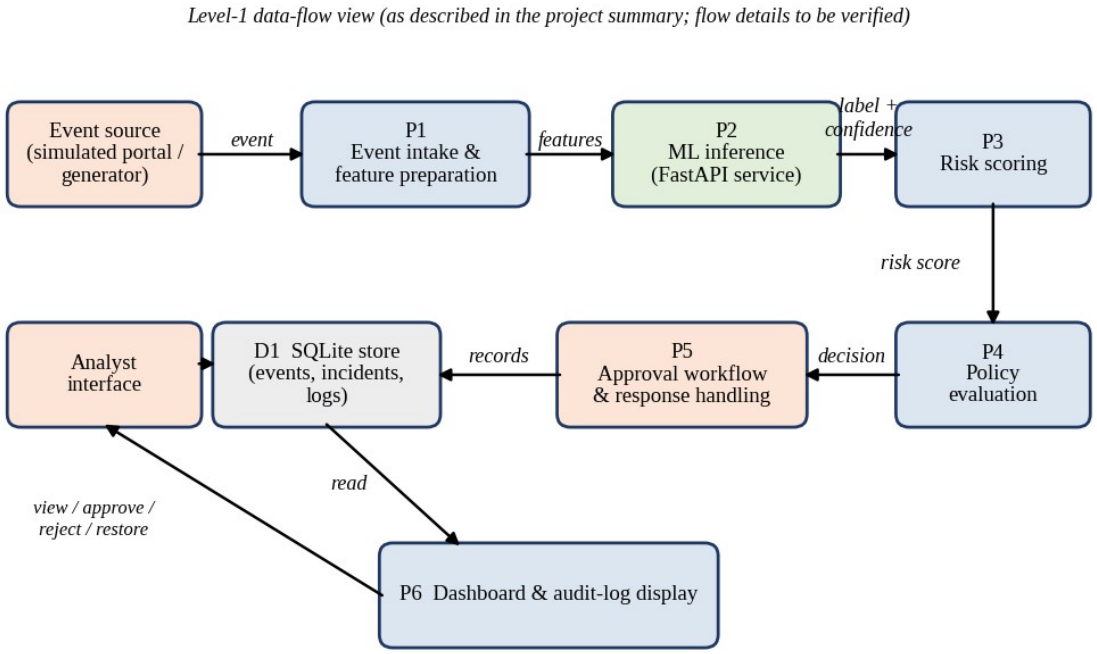
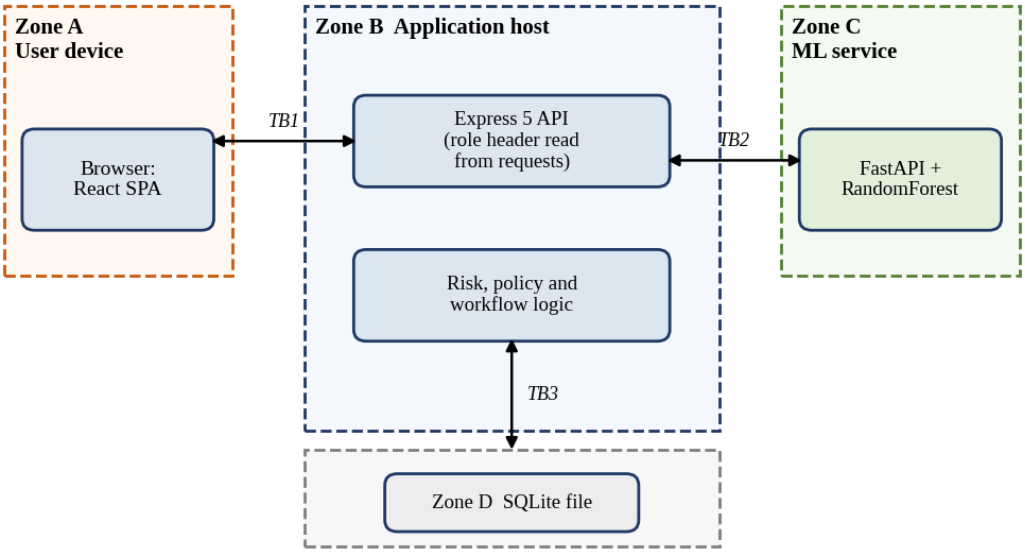



**Figure 4.3 Level-1 Data-Flow Diagram of the Orbit AI Prototype**

## 4.10 TRUST BOUNDARIES AND SECURITY CONSIDERATIONS

Figure 4.4 identifies three trust boundaries that matter for the prototype. The first lies between the user's browser and the back end. The project summary states that a user-role request header may be used for authorisation rather than a verified authentication mechanism. A client can set such a header to any value, so it cannot be relied on to establish identity or role. The second lies between the back end and the ML service, where service-to-service authentication has not been confirmed. The third lies between the back end and the database, where start-up behaviour and data protection need review.




TB1 Browser to API: the user role is taken from a request header (not a verified identity).
TB2 API to ML service: service-to-service authentication is not confirmed.
TB3 API to database: persistence and start-up behaviour require review.

**Figure 4.4 Trust Boundaries of the Orbit AI Prototype**

These considerations mean that the prototype must be run only in a controlled demonstration environment. It should not be exposed to untrusted networks or used with real accounts or real security data. Credentials and secrets are not reproduced in this report.

## 4.11 VERIFICATION STATUS OF IMPLEMENTATION CLAIMS

Table 4.5 lists the principal technical claims made in this report, the source of each claim and the evidence that must be checked in the repository. The status column is deliberately conservative: an item is shown as verified only after it has been matched to the source code.

| Claim | Where to verify | Status |
|---|---|---|
| React 19 and Vite front end | Front-end package manifest | To be verified |
| Node.js and Express 5 back end | Back-end package manifest | To be verified |
| SQLite with Sequelize | Back-end models and database configuration | To be verified |
| FastAPI ML service with Scikit-learn | ML service requirements and entry point | To be verified |
| 100 trees, depth 8, five features, five classes | Training script | To be verified |
| About 3,000 synthetic training records | Training script / data generator | To be verified |

| Claim | Where to verify | Status |
|---|---|---|
| Confidence from maximum predict_proba | Prediction function | To be verified |
| Risk factors and weights | Risk-scoring module | To be verified |
| Asset criticality set to HIGH for alerts | Alert intake code | To be verified |
| Policy rules and approval-required actions | Policy module | To be verified |
| Google GenAI not operational in the workflow | Search for imports and calls | To be verified |
| Known limitations KL-1 to KL-10 | See Table 9.1 | Open until re-checked |

**Table 4.5 Implementation Claims and Verification Status**

## 4.12 THREAT CONSIDERATIONS FOR THE PROTOTYPE ITSELF

Because Orbit AI is a security tool, its own weaknesses matter. Table 4.6 lists threats to the prototype, their current status as understood from the preliminary review and the type of mitigation that would be required. None of the mitigations is claimed as implemented.

| Threat | Description | Current status | Mitigation (future) |
|---|---|---|---|
| Role spoofing | A client sets a role header to gain privileged actions | Possible (KL-8) | Verified authentication and RBAC |
| Unauthorised approval | Approval or rejection by an unauthorised user | Depends on KL-8 | Server-side authorisation checks |
| Input tampering | Manipulated features sent to the ML service | Not assessed | Input validation and service authentication |
| Data loss | Database reset at start-up | Possible (KL-6) | Remove forced synchronisation; backups |
| Over-trust in confidence | Confidence read as probability of attack | Mitigated in text only | Calibration and clear display wording |
| Incomplete records | Missing audit entries | Possible (KL-9) | Log all workflow stages |
| Secret exposure | Credentials in configuration or code | Not assessed | Secret management and review |

**Table 4.6 Threats to the Prototype and Required Mitigations**

# CHAPTER 5

# METHODOLOGY AND IMPLEMENTATION

## 5.1 OVERVIEW OF THE METHODOLOGY

The methodology combines a supervised machine-learning classifier with deterministic engines and a human decision step. The machine-learning model answers a single question: given the features of an event sequence, which of five categories does it most resemble? Everything else in the workflow – how serious the situation is, what should be done and who must approve it – is governed by deterministic logic or by the analyst. This separation is the main design principle of the prototype.

## 5.2 STEP-BY-STEP WORKFLOW

The workflow consists of eleven steps, shown in Figure 5.1 and described in Table 5.1.

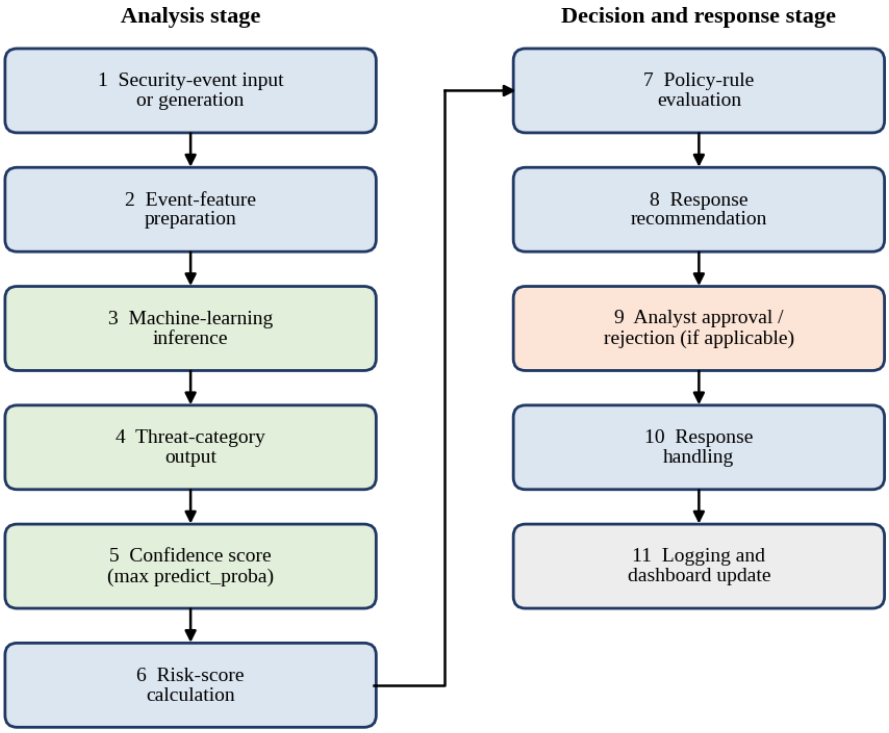



**Figure 5.1 Event-Processing Workflow of Orbit AI**

| No. | Step | Description | Performed by |
|---|---|---|---|
| 1 | Security-event input or generation | Events are submitted through the simulated portal or generated for demonstration. | User / generator |
| 2 | Event-feature preparation | A sequence of events is summarised into five numerical features. | Back end |
| 3 | Machine-learning inference | The feature vector is sent to the ML service and passed to the Random Forest. | ML service |

| No. | Step | Description | Performed by |
|---|---|---|---|
| 4 | Threat-category output | The model returns one of five category labels. | ML service |
| 5 | Confidence-score calculation | The maximum class probability from predict_proba is used as the confidence. | ML service |
| 6 | Risk-score calculation | The risk engine combines base severity, confidence, asset criticality and repeated events. | Risk engine |
| 7 | Policy-rule evaluation | Deterministic rules are applied to the category and risk. | Policy engine |
| 8 | Response recommendation | A response action is selected or controlled by policy. | Policy engine |
| 9 | Analyst approval or rejection | For selected actions, the analyst approves or rejects. | Analyst |
| 10 | Response handling | The approved or permitted action is handled in the prototype. | Back end |
| 11 | Logging and dashboard update | Outcomes are recorded and displayed. | Back end / front end |

**Table 5.1 Description of the Eleven Workflow Steps**

## 5.3 INPUT FEATURES

Each event sequence is represented by a vector of five integer-valued features. These features were chosen because they can be counted from simple authentication and access events and because they relate directly to the behaviours that the five categories describe.

| No. | Feature | Meaning |
|---|---|---|
| 1 | Failed logins | Number of unsuccessful authentication attempts in the sequence. |
| 2 | Successful logins | Number of successful authentication attempts in the sequence. |
| 3 | Sensitive access | Number of accesses to resources designated as sensitive. |
| 4 | Privilege activity | Number of privilege-related actions, such as privilege changes. |
| 5 | Sequence length | Total number of events in the analysed sequence. |

**Table 5.2 Input Features of the Classification Model**

## 5.4 SYNTHETIC TRAINING DATA

The model is trained on approximately 3,000 synthetic records generated by the project's training pipeline. Synthetic data was used because a labelled enterprise event dataset was not available to the project. The records are produced from rules or distributions chosen by the developers for each class, so the class labels reflect the developers' assumptions about what each pattern looks like.

This has important consequences. First, a model trained and tested on data from the same generator will tend to learn the generator's assumptions and may appear to perform well simply because the classes were designed to be separable. Second, such a result says little about how the model would behave on real events, in which classes overlap, counts are noisy and behaviour changes over time. Third, the model has not been calibrated or tested against an independent labelled set. These points follow the cautions in the literature on applying machine learning to intrusion detection [11], and they are why the report makes no claim of real-world detection performance.

## 5.5 THE RANDOM FOREST ALGORITHM

### 5.5.1 Concept

A Random Forest is an ensemble of decision trees. A single decision tree classifies an input by asking a sequence of threshold questions about the features, such as whether the number of failed logins exceeds a value, until it reaches a leaf that assigns a class. Individual trees tend to fit the training data too closely. A Random Forest reduces this effect by training many trees on different random samples of the training data and by letting each split consider only a random subset of the features. The predictions of the trees are then aggregated. Orbit AI uses the RandomForestClassifier implementation in Scikit-learn [12].

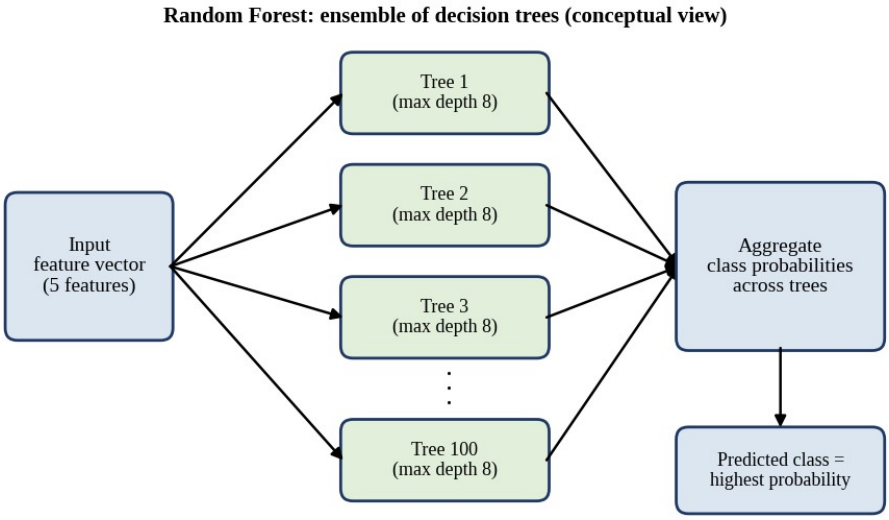



**Figure 5.2 Conceptual View of the Random Forest Ensemble**

### 5.5.2 Training workflow

1. Synthetic records with five features and a class label are generated by the training pipeline.
2. For each of the 100 trees, a bootstrap sample of the records is drawn.
3. Each tree is grown by repeatedly choosing the split that best separates the classes (for example using an impurity measure) among a random subset of the features, until the maximum depth of 8 or another stopping condition is reached.
4. The trained forest is made available to the FastAPI service for prediction.

### 5.5.3 Prediction process

For a new feature vector, each tree produces a class distribution. The forest averages these distributions to obtain the class probabilities returned by predict_proba, and the class with the highest averaged probability is the predicted category. The overall inference pipeline is shown in Figure 5.3.

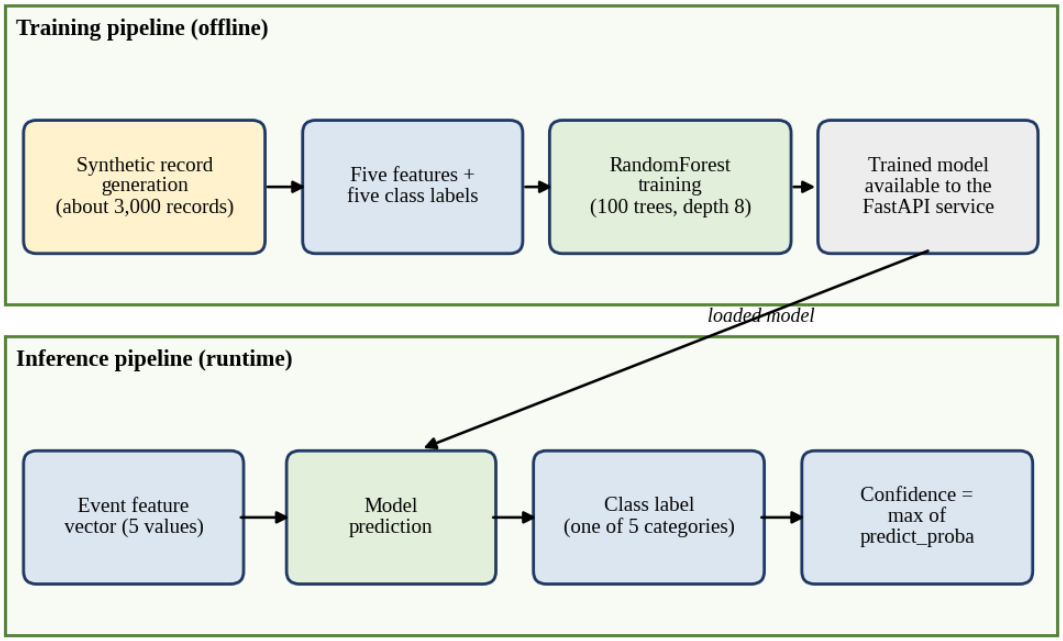



**Figure 5.3 Machine-Learning Training and Inference Pipeline**

### 5.5.4 Configuration

| Parameter | Value |
|---|---|
| Algorithm | RandomForestClassifier (Scikit-learn) |
| Number of trees | 100 |
| Maximum tree depth | 8 |
| Input features | 5 |
| Output classes | 5 |
| Training data | About 3,000 synthetic records generated by the training pipeline |

**Table 5.3 Model Configuration**

### 5.5.5 Limitations of the algorithm in this setting

- A Random Forest learns only from the patterns present in its training data; behaviour outside that distribution may be classified confidently but wrongly.
- The averaged tree probabilities are not guaranteed to be calibrated probabilities.
- With five coarse count features, different real-world behaviours can map to the same vector and cannot be distinguished.

- The model has no memory of earlier sequences, so temporal relationships beyond the counts are not represented.

## 5.6 THREAT CLASSIFICATION CATEGORIES

| Category | Intended meaning in the prototype |
|---|---|
| Monitoring | Activity that does not match a threat pattern and is kept under observation. |
| Authentication Anomaly | Authentication behaviour that departs from the expected pattern, for example unusual numbers of failed logins. |
| Account Compromise | A pattern that suggests an account may have been taken over, for example failures followed by success and subsequent activity. |
| Data Collection | A pattern dominated by access to sensitive resources. |
| Privilege Escalation | A pattern that includes notable privilege-related activity. |

**Table 5.4 Threat Classification Categories**

The descriptions in Table 5.4 are explanatory summaries of the category names. The exact rules used to generate each class in the synthetic data must be taken from the training pipeline and added when the source is verified.

## 5.7 CONFIDENCE SCORE

The confidence value is the maximum class probability returned by predict_proba for the input. It expresses how strongly the trees favour the winning class. It is a model confidence score. It is not a calibrated probability that a real-world cyberattack is occurring, because calibration has not been independently demonstrated. A high confidence on an input unlike the training data is possible. The confidence is used as one input to risk scoring and is displayed to the analyst, but it should be interpreted with these limits in mind.

## 5.8 RISK ASSESSMENT ENGINE

The risk-scoring engine is deterministic: it contains no learned parameters, and identical inputs always give the same score. As described in the project, it uses factors such as the base severity associated with the predicted category, the model confidence and the asset criticality, and repeated events may increase the score. Figure 5.4 shows how these inputs feed the engine and the policy engine.

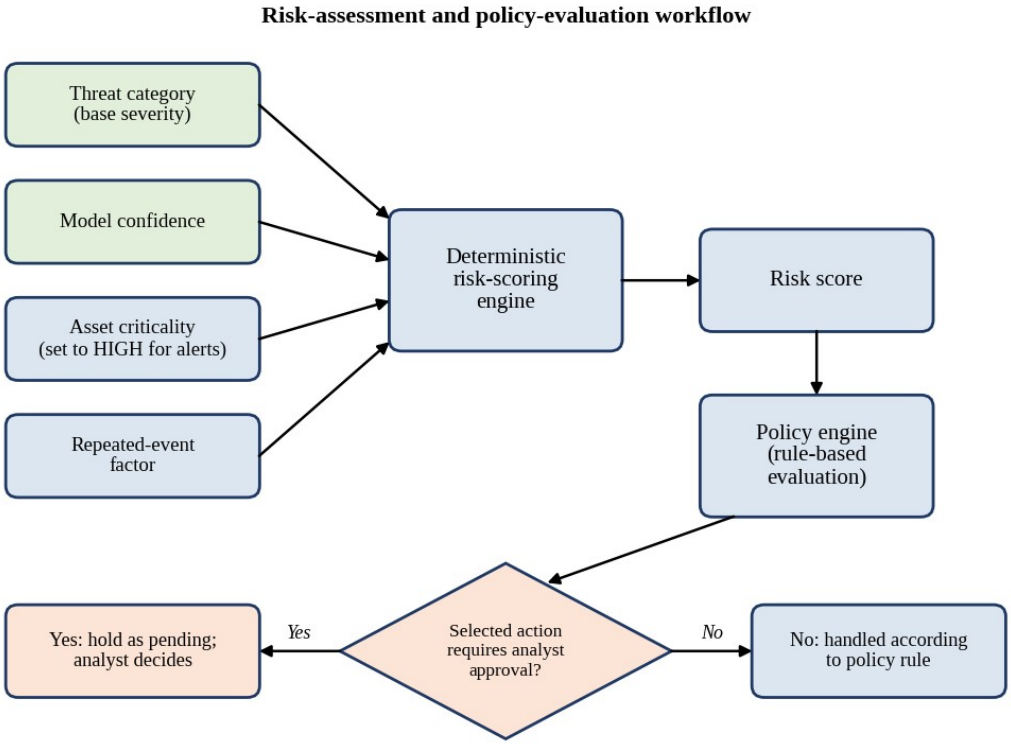



**Figure 5.4 Risk-Assessment and Policy-Evaluation Workflow**

In the current implementation the asset criticality of an incoming alert is set to HIGH. The prototype therefore does not distinguish between critical and non-critical assets, and the criticality factor should not be presented as a dynamic enterprise asset-management feature. The Risk Analysis screen in Figure 6.6 displays the target asset classification as Production Endpoint (Tier 2), which is consistent with a fixed rather than dynamic value. The numerical weights and thresholds of the engine are to be recorded in Appendix D from the source code.

## 5.9 POLICY ENGINE

The policy engine is rule-based logic. It evaluates conditions on the category, risk score and related state and produces a decision about the response action, including whether the action requires analyst approval. It does not reason autonomously, plan or learn, and it should not be described as an advanced AI agent. Its advantage is predictability: the rules can be read, tested and changed. Its disadvantage is that it handles only situations anticipated by its authors. The rules actually implemented are to be listed in Appendix D.

## 5.10 RESPONSIBILITIES OF EACH COMPONENT

| Component | Responsibility | Does not do |
|---|---|---|
| ML model | Assigns a category and confidence to a feature vector | Decide actions or assess asset importance |
| Risk engine | Computes a numerical risk score by fixed logic | Learn from data or adapt weights |
| Policy engine | Selects or controls actions by explicit rules | Reason autonomously or plan |
| Analyst | Approves or rejects selected actions; restores accounts | Is not replaced by the system for selected actions |
| Back end and database | Coordinates steps and stores records | Guarantee audit completeness (to be verified) |

**Table 5.5 Division of Responsibilities in the Orbit AI Workflow**

## 5.11 ANALYST APPROVAL AND RESPONSE HANDLING

When policy selects an action that requires approval, the action is held in a pending state until an analyst decides. The analyst reviews the category, confidence, risk score and recommendation, and then approves or rejects the action. An approved action proceeds under the prototype's response handling, and a rejected action is not executed. The analyst can also restore an account that has been affected by a response. Figure 5.5 shows this workflow. The outcome of each decision is recorded; the completeness of the recorded information is a known concern (Chapter 9).

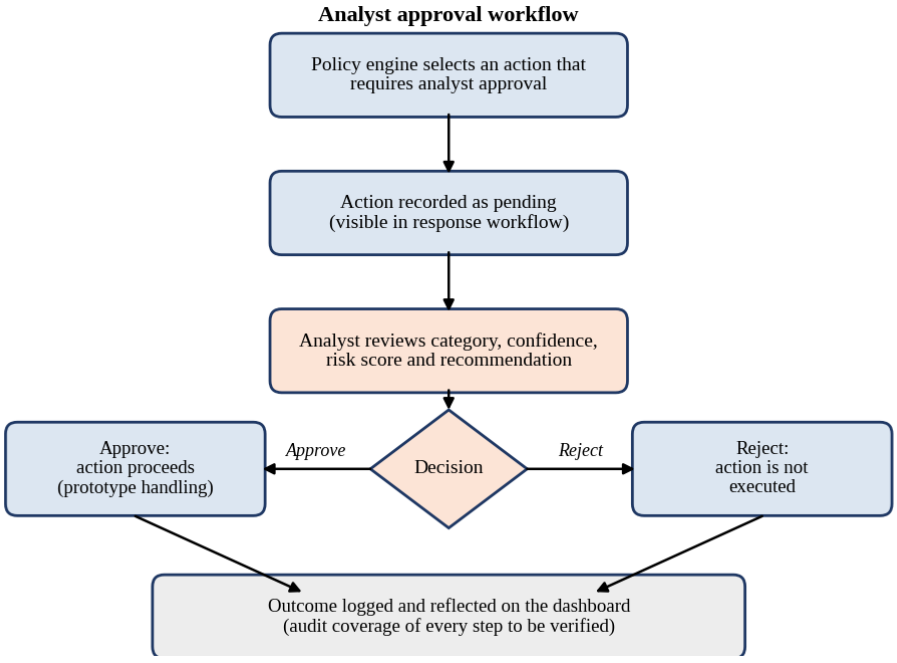



**Figure 5.5 Analyst Approval Workflow**

All response actions in the prototype operate within the demonstration environment on simulated accounts. The prototype is not connected to real enterprise systems.

## 5.12 DETERMINISTIC AUTOMATION VERSUS GENERATIVE AI

A Google GenAI package may be listed among the project's dependencies. However, this report does not claim that the system performs Gemini-powered reasoning or orchestrates large-language-model agents, because the presence of a dependency does not show that the integration is implemented and operational. The classification is performed by a Random Forest, and the risk and policy decisions are made by deterministic code. Unless a code inspection shows otherwise, the automation in Orbit AI should be understood as deterministic rule-based automation combined with a conventional machine-learning classifier, and not as generative-AI functionality. The interface of the prototype uses labels such as Agentic Response Workflow and Agent Engine Active (Figure 6.7); these are names of interface sections and are not evidence of large-language-model reasoning.

## 5.13 LOGGING AND DASHBOARD UPDATE

Workflow activity is stored in the database and presented through the dashboard and the audit-log interface. A preliminary review suggested that the audit trail may not capture every initial event, inference and policy decision. Until this is verified and addressed, the audit logs should be regarded as partial.

## 5.14 SUITABILITY CONSIDERATIONS FOR THE DESIGN CHOICES

The following considerations explain why the chosen techniques are suitable for a demonstration prototype. They are not claims that the choices are optimal.

- **Random Forest.** It works with small tabular datasets, handles non-linear relationships between count features, trains quickly and is available in Scikit-learn [12]. It also returns class probabilities from which a confidence value can be derived.
- **Separate ML service.** The Python machine-learning ecosystem and the Node.js web ecosystem can be used together, and the model can be replaced without changing the web application.
- **Deterministic risk and policy engines.** Their behaviour can be read and unit-tested, which a learned model alone cannot offer.
- **SQLite.** It requires no separate database server, which simplifies demonstration, although it limits scalability.
- **Analyst approval.** It places a human decision before selected actions, which limits the effect of a misclassification.

# CHAPTER 6

# SYSTEM MODULES AND USER INTERFACE

## 6.1 INTRODUCTION

This chapter documents the screens and modules of the Orbit AI prototype. Each module is described by its purpose, input, processing, output and, where the project description supports it, its interaction with the back end and database. The screenshots in this chapter are real captures of the running prototype, taken on 10 October 2026 in the SECURITY MONITORING environment with the user role shown as SOC_ANALYST. They come from a single demonstration session in which the same simulated user (demo_user), the same source address (192.168.1.55) and the same incident (ALT-8329) appear throughout.

The screenshots show what the interface displayed at the time of capture. They do not by themselves prove how a value was computed, so statements about internal processing are limited to what the project description supports, and details that still have to be confirmed against the source code are shown as highlighted placeholders. Several observations that matter for interpreting the results are collected in Chapters 8 and 9.

| Menu Item | Screen Title | Figure |
|---|---|---|
| Overview | Security Operations Center | Figure 6.1 |
| Security Events | Security Event Telemetry | Figure 6.2 |
| Threat Detection | ML Threat Detection Pipeline | Figure 6.3 |
| Incidents | Active Incident Queue | Figure 6.4 |
| Attack Paths | Path Analysis Engine | Figure 6.5 |
| Risk Analysis | Assessment Report | Figure 6.6 |
| Recommendations | Agentic Response Workflow | Figure 6.7 |
| Accounts & Recovery | Account Security Handlers | Figure 6.8 |
| Open Demo App (button) | Acme Corp Portal (simulated) | Figure 6.9 |

**Table 6.1 Navigation Structure of the Orbit AI Interface**

## 6.2 SECURITY OPERATIONS DASHBOARD

The Overview screen is the main view of the analyst. As captured in Figure 6.1, it shows three counters (Total Events 10, Detected Threats 1 and Active Incidents 1), an Engine Health and Diagnostics panel, and a Recent Telemetry table with timestamp, source IP, user, event type and status. The header displays the environment, an Open Demo App button, a MONITORING status badge and a Fast-Forward 5 Min button.

The Engine Health panel reports the Predictive Engine as ACTIVE, the Risk Engine as ACTIVE and the Database as CONNECTED, but the ML Engine as FALLBACK. This observation is important. It indicates that, at the time of capture, the machine-learning component was operating in a fallback state, so the categories and confidence values displayed elsewhere cannot be assumed to have been produced by the Random Forest model. What the fallback consists of must be confirmed from the source code. The Fast-Forward control was reported in a preliminary review to depend on a back-end interface that may not exist (KL-5); the capture shows only that the button is present.

| | |
|---|---|
| **Purpose** | Summarise event counts, detected threats, active incidents and engine health for the analyst. |
| **Input** | Event, incident and health data retrieved from the back end; analyst interaction. |
| **Processing** | The React interface retrieves and renders the data. The refresh method and the source of each health indicator are [to be completed from the source]. |
| **Output** | Counters, engine-health indicators and recent telemetry (Figure 6.1). |
| **Back end / database** | Reads event and incident data through the back end API [endpoints to be confirmed]; the Database indicator showed CONNECTED. |

**Table 6.2 Security Operations Dashboard Module**

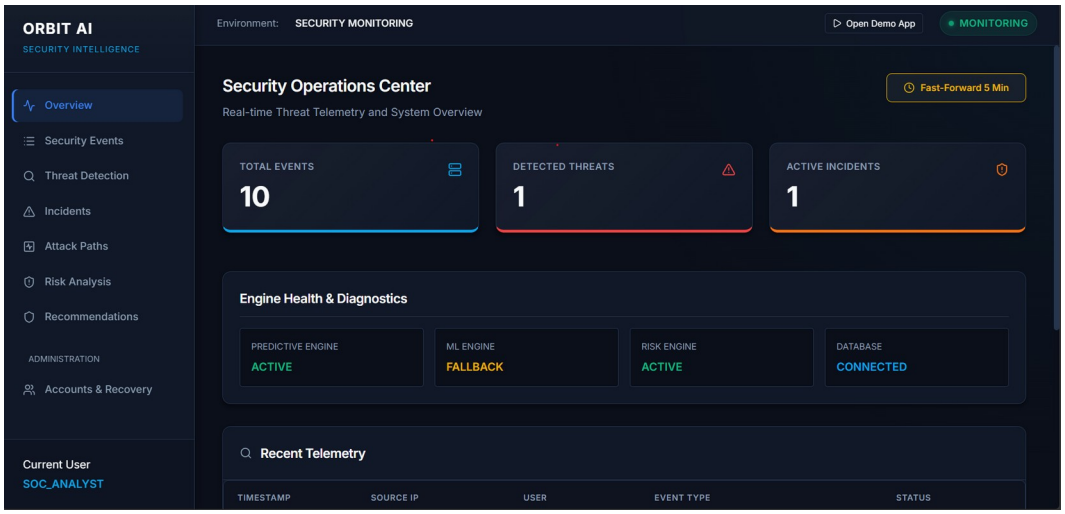



**Figure 6.1 Security Operations Dashboard (Overview Screen)**

## 6.3 SECURITY EVENTS MODULE

The Security Events screen, titled Security Event Telemetry, lists raw event logs with a search box and filters for event type and status. In Figure 6.2 the visible rows are all for demo_user from 192.168.1.55, with event types FILE_ACCESS and SENSITIVE_FILE_ACCESS, status SUCCESS and a target resource of N/A. The visible timestamps run from 9:21:09 pm to 9:22:27 pm on 10/10/2026. The target-resource column shows N/A for every visible row, so that field does not appear to be populated for these events.

| | |
|---|---|
| **Purpose** | Display raw security events so that the analyst can search and filter them. |
| **Input** | Stored events; search text, event-type filter and status filter. |
| **Processing** | Filtering of the event list on the client or server [to be confirmed]. |
| **Output** | Table of timestamp, event ID, entity/IP, user, event type, target resource and status. |
| **Back end / database** | Reads stored events [table name to be confirmed]. |

**Table 6.3 Security Events Module**

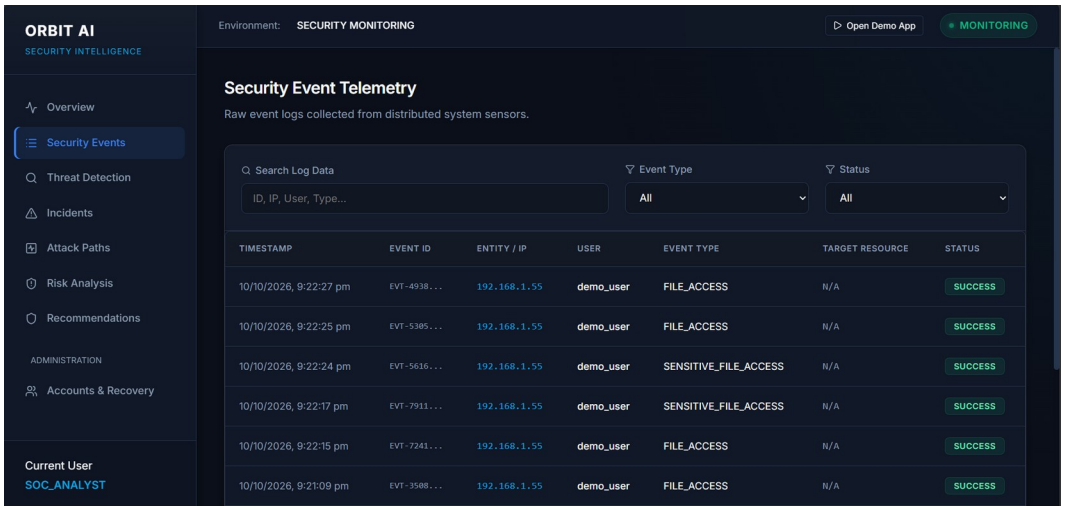



**Figure 6.2 Security Event Telemetry Screen**

## 6.4 THREAT DETECTION MODULE

The Threat Detection screen, titled ML Threat Detection Pipeline, lets the analyst run manual anomaly detection on unclassified events. As captured in Figure 6.3, it contains an Unanalyzed Events Queue selector, an Execute ML Pipeline button and a result panel that reads “Select an event to run through the detection pipeline”. The capture shows the screen before any event is selected, so no detection output is shown. A preliminary review reported that the manual inference interface may call a stub that returns a misleading benign result (KL-3). Because the ML engine was in FALLBACK at the time of capture, the interface must be tested against the real ML service before its output is relied on.

| | |
|---|---|
| **Purpose** | Run the classification pipeline manually on a selected, unanalysed event. |
| **Input** | An event chosen from the unanalysed-events queue. |
| **Processing** | Features are prepared and sent to the ML service, which returns a category and class probabilities; confidence is the maximum probability (Chapter 5). The actual call path of this screen is [to be confirmed]. |
| **Output** | Detection result for the selected event; none is shown in Figure 6.3. |
| **Back end / database** | Back-end request to the ML service; resulting incident stored [to be confirmed]. |

**Table 6.4 Threat Detection Module**

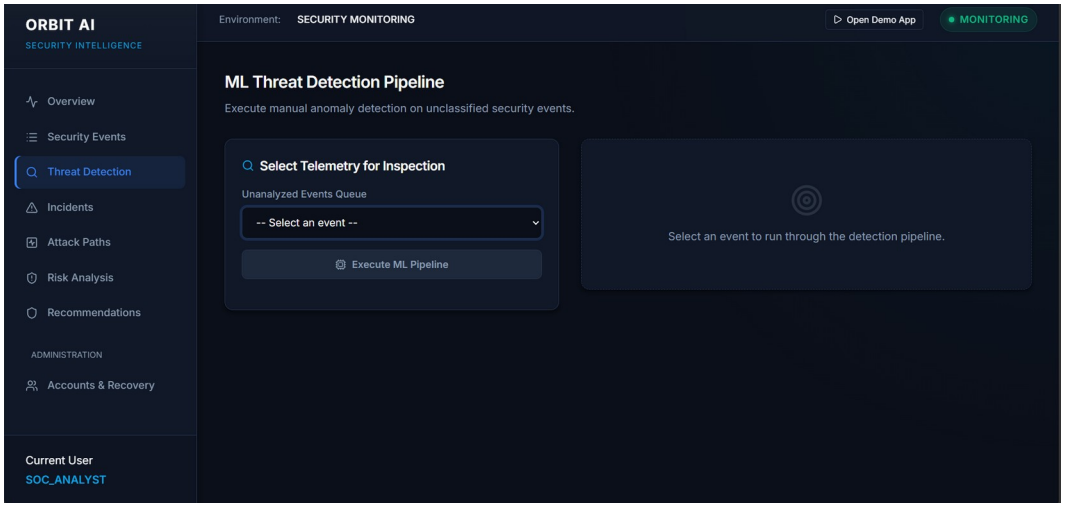



**Figure 6.3 ML Threat Detection Pipeline Screen**

## 6.5 INCIDENT QUEUE MODULE

The Incidents screen, titled Active Incident Queue, lists alerts generated by the detection engine. Figure 6.4 shows one incident, ALT-8329, created on 10/10/2026 at 9:21:08 pm. Its threat vector is Suspicious Account Activity, the origin IP is 192.168.1.55, the current stage is Sensitive Data Access and the predicted stage is Data Collection. The risk is shown as HIGH (100), the confidence as 88%, the deviation as NO and the status as OPEN. The confidence value is the model confidence described in Section 5.7 and is not a calibrated probability of an attack. The risk label HIGH (100) differs from the label CRITICAL for the same incident in Figure 6.6, which indicates inconsistent labelling of the score.

| | |
|---|---|
| **Purpose** | List incidents with risk, confidence and status, ordered by severity. |
| **Input** | Incident records created by the detection and risk workflow. |
| **Processing** | Retrieval and display of stored incidents; ordering by severity as stated on the screen. |
| **Output** | Incident ID, timestamp, threat vector, origin IP, current stage, predicted stage, risk, confidence, deviation and status. |
| **Back end / database** | Reads the incident table [name to be confirmed]. |

**Table 6.5 Incident Queue Module**

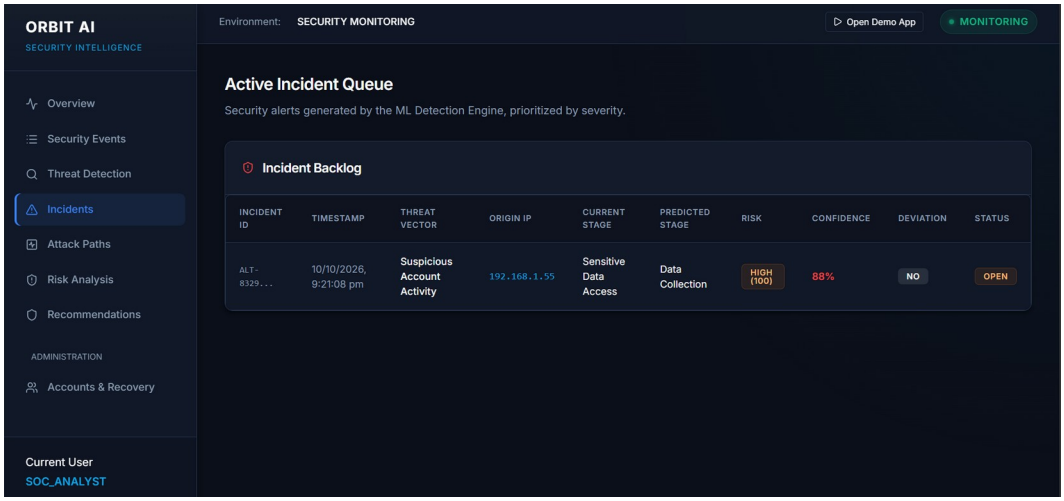



**Figure 6.4 Active Incident Queue Screen**

## 6.6 ATTACK PATHS MODULE

The Attack Paths screen presents a Path Analysis Engine for a selected incident (ALT-8329, Suspicious Account Activity). Figure 6.5 shows a vertical chain of Observed Behavior steps – Authentication Anomaly, Account Access and Sensitive Data Access – followed by Predicted Next Move steps beginning with Further Data Collection. The capture is cut off below the second predicted step. The method by which next moves are predicted is not described in the supplied material and is [to be confirmed from the source code]. This report therefore describes the screen only as a display of observed and predicted stages and makes no claim of forecasting performance.

| | |
|---|---|
| **Purpose** | Show the sequence of observed behaviours for an incident and the predicted next stages. |
| **Input** | A selected incident. |
| **Processing** | Derivation of observed and predicted stages [method to be confirmed]. |
| **Output** | Ordered chain of observed and predicted steps. |
| **Back end / database** | Reads incident and event data [to be confirmed]. |

**Table 6.6 Attack Paths Module**

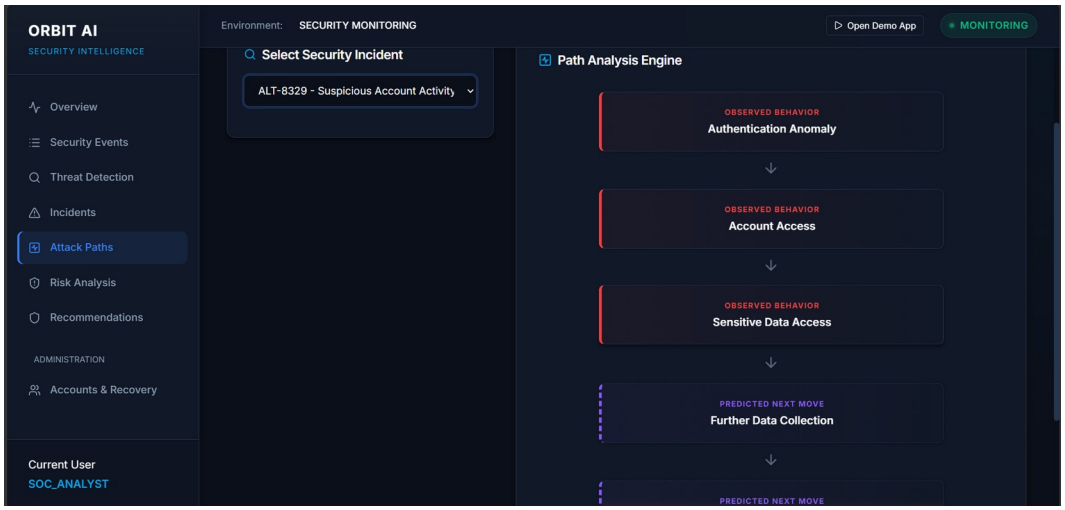



**Figure 6.5 Attack Paths Screen for Incident ALT-8329**

## 6.7 RISK ASSESSMENT MODULE

The Risk Analysis screen shows an Assessment Report for the selected incident. In Figure 6.6 the calculated risk score is 100/100 and the aggregate risk level is CRITICAL. The calculation factors listed are Initial Threat Severity HIGH, Event Sequence Correlation “Verified Pattern (Deterministic)”, Potential Business Impact “System Compromise & Privilege Escalation” and Target Asset Classification “Production Endpoint (Tier 2)”. The Analyst Priority is Immediate Review Required (P1). The listed factors are broadly consistent with the factors described for the deterministic engine, but the weights and formula are [to be confirmed], and a single score of 100 does not show how the score varies across incidents.

| | |
|---|---|
| **Purpose** | Convert the incident context into a numerical risk score, a risk level and an analyst priority. |
| **Input** | Selected incident; base severity, model confidence, asset classification and repeated-event information. |
| **Processing** | Deterministic scoring logic with no learned parameters; weights [to be listed in Appendix D]. |
| **Output** | Risk score (100/100), risk level (CRITICAL), calculation factors and analyst priority (P1). |
| **Back end / database** | Computed in the back end [to be confirmed]; the Risk Engine indicator showed ACTIVE. |

**Table 6.7 Risk Assessment Module**

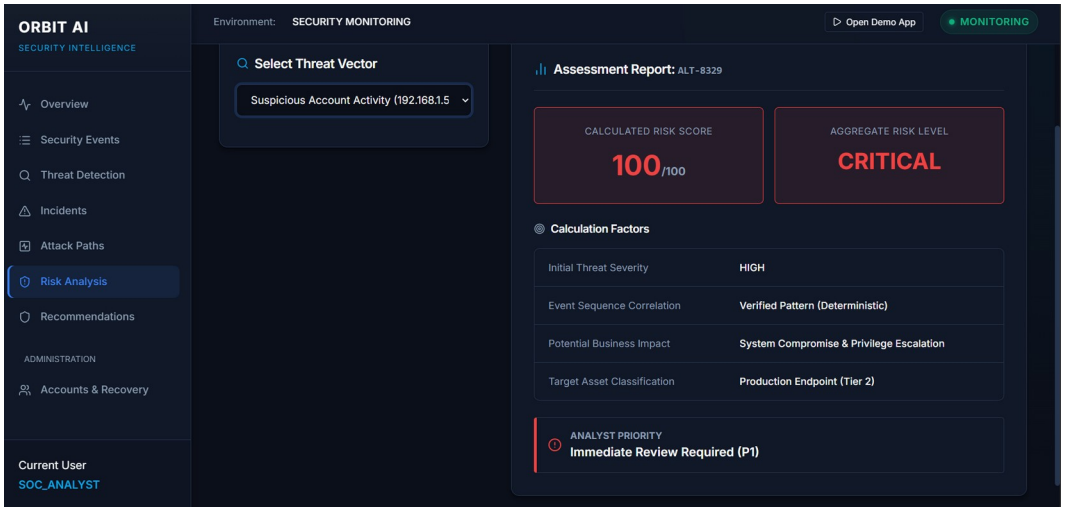



**Figure 6.6 Risk Analysis Screen for Incident ALT-8329**

## 6.8 RESPONSE WORKFLOW AND ANALYST APPROVAL MODULE

The Recommendations screen is titled Agentic Response Workflow and carries an AGENT ENGINE ACTIVE badge. In Figure 6.7 it presents one item, Suspicious Account Activity (event reference EVT-2508, entity 192.168.1.55), with the status badge APPROVED, a reasoning message, a Recommended Action field and two controls, Approve & Mitigate and Reject. The reasoning message states that a sequence of events matching known malicious patterns was identified and that automated mitigation is recommended to prevent further lateral movement.

Three observations should be noted. First, the Recommended Action field shows an empty coloured chip, so the recommended action itself is not readable; this is consistent with the reported possibility that the interface reads the wrong recommendation properties (KL-4). Second, the badge reads APPROVED while the approval and rejection controls are still displayed, so the relation between the badge and the controls needs to be clarified. Third, the words Agentic and Agent Engine are interface labels. Consistent with Section 5.12, this report does not treat them as evidence of large-

language-model reasoning: the reasoning message appears to be a general statement, and whether it is generated or fixed text is [to be confirmed from the source code].

| | |
|---|---|
| **Purpose** | Show the recommended response to an incident and let the analyst approve or reject it. |
| **Input** | Policy decision for the incident; analyst choice (Approve & Mitigate or Reject). |
| **Processing** | Rule-based policy selects the action; approval proceeds the action and rejection prevents it (Section 5.11). Role checking uses a request header, which is not robust authorisation (KL-8). |
| **Output** | Status badge, reasoning message, recommended action (blank in Figure 6.7) and decision controls. |
| **Back end / database** | Approval state update and associated record [to be confirmed]. |

**Table 6.8 Response Workflow and Analyst Approval Module**

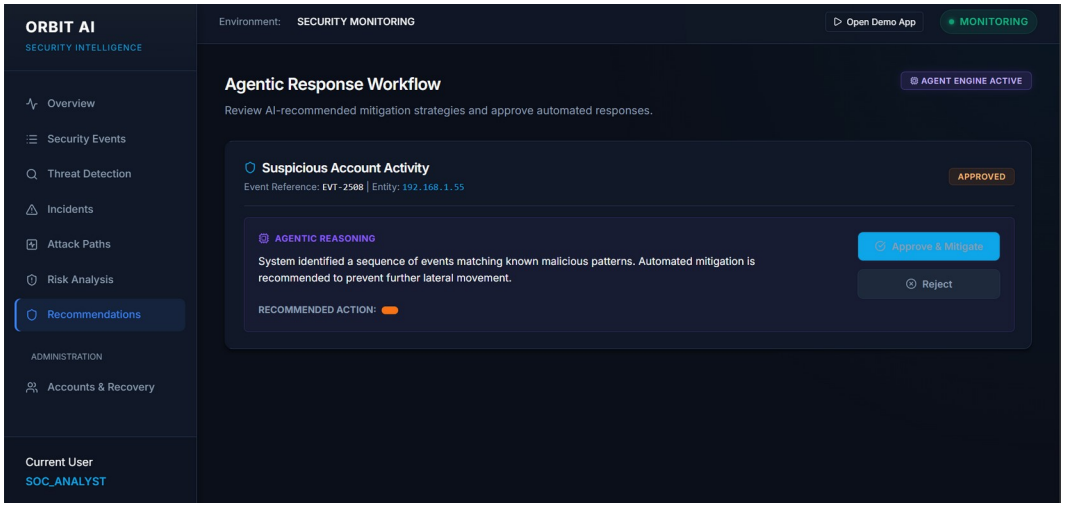



**Figure 6.7 Agentic Response Workflow Screen (Recommendations)**

## 6.9 ACCOUNT RESTORATION MODULE

The Accounts & Recovery screen, titled Account Security Handlers, lets the analyst review escalated incidents and manage access for restricted accounts. Figure 6.8 shows demo_user with the status BLOCKED, the escalated incident ID ALT-8329, and an automated-reasoning message stating that repeated suspicious activity was detected and that the account was escalated to the BLOCKED state after a cooldown violation. The block time is shown as 10/10/2026, 9:27:03 pm, and a Restore Access button is provided. The capture shows that an account restriction was applied in the demonstration and that a manual restore control exists; it does not show the restoration itself. The cooldown rule is not defined in the supplied material and is to be documented from the source code.

| | |
|---|---|
| **Purpose** | Review restricted accounts and restore access manually. |
| **Input** | Escalated incident and the analyst's Restore Access request. |
| **Processing** | The account status changes from BLOCKED to restored [exact states to be confirmed]. |
| **Output** | Updated account status and a record of the action [to be confirmed]. |
| **Back end / database** | Account status record [table name to be confirmed]. |

**Table 6.9 Account Restoration Module**

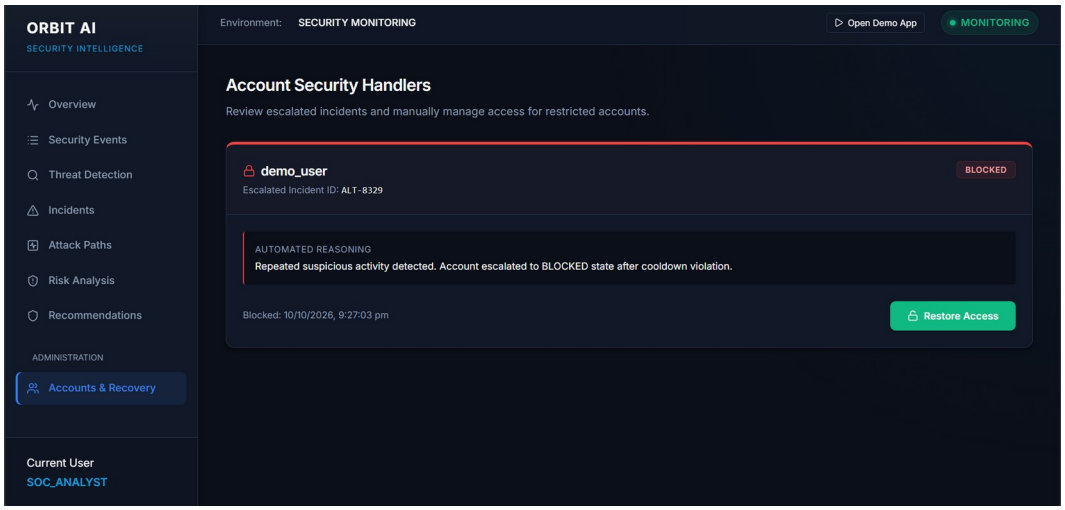



**Figure 6.8 Accounts and Recovery Screen**

## 6.10 SIMULATED ENTERPRISE PORTAL

The simulated enterprise portal, opened through the Open Demo App button, is a demonstration environment named Acme Corp Portal. Figure 6.9 shows the My Files page for the user demo_user with five files and their classifications: project-report.pdf, team-notes.txt and development-guide.pdf (NORMAL), employee-data.csv (CONFIDENTIAL) and financial-records.csv (HIGHLY CONFIDENTIAL). Each file has an Open button. The presence of FILE_ACCESS and SENSITIVE_FILE_ACCESS events for demo_user in Figure 6.2 suggests that opening files in the portal generates events, but the mechanism is [to be confirmed]. The portal is not a real enterprise system and processes no real user data.

| | |
|---|---|
| **Purpose** | Provide a demonstration source of user activity and events. |
| **Input** | Simulated user actions such as logging in and opening files of different classifications. |
| **Processing** | Actions are turned into events for the analysis workflow [mechanism to be confirmed]. |
| **Output** | Portal pages and, through the analysis workflow, events, incidents and responses. |
| **Back end / database** | Event submission to the back end [to be confirmed]. |

**Table 6.10 Simulated Enterprise Portal Module**

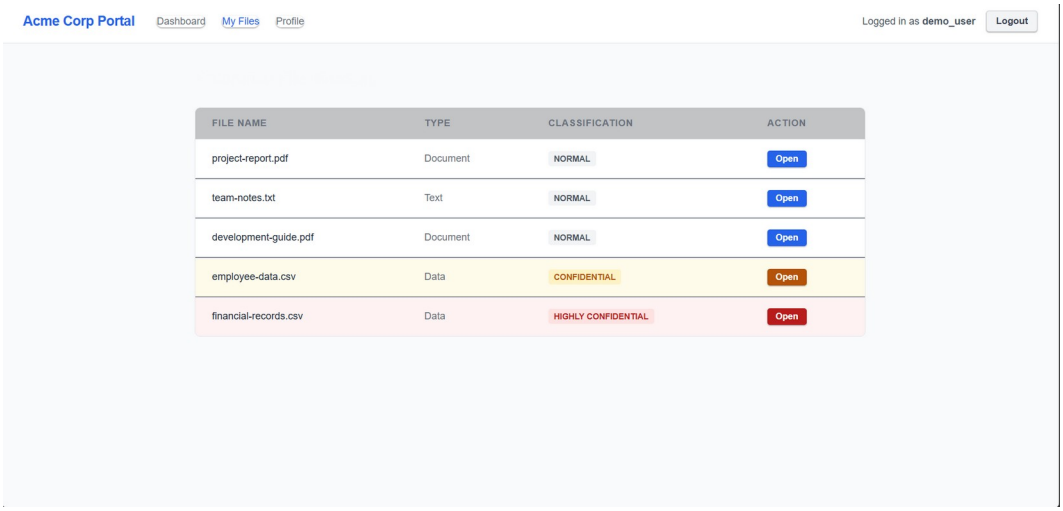



**Figure 6.9 Simulated Enterprise Portal (Acme Corp Portal, My Files)**

## 6.11 AUDIT LOGGING

Audit logging is part of the project description, but the navigation menu in Figure 6.1 contains no dedicated audit-log screen, and none was supplied as a screenshot. The Security Events screen displays raw telemetry, which is not the same as an audit trail of inferences, policy decisions and analyst actions. Whether an audit-log interface exists, and what the audit records contain, is therefore [to be confirmed from the source code], and this remains linked to limitation KL-9.

| | |
|---|---|
| **Purpose** | Provide a record of workflow activity for review. |
| **Input** | Records created by the back end during event processing and analyst actions. |
| **Processing** | Storage of audit records; a preliminary review suggested that some stages may not be recorded (KL-9). |
| **Output** | No dedicated screen appears in the supplied screenshots. |
| **Back end / database** | Audit table [name to be confirmed]. |

**Table 6.11 Audit Logging Module**

## 6.12 SUMMARY OF MODULE DEPENDENCIES AND KNOWN ISSUES

| Module | Main Dependency | Related Issue | Figure |
|---|---|---|---|
| Dashboard | Back-end data API | ML Engine shown as FALLBACK; KL-5 | 6.1 |
| Security events | Stored events | Target resource N/A | 6.2 |
| Threat detection | ML service via back end | KL-3, KL-1 | 6.3 |
| Incident queue | Incident records | Risk label differs from 6.6 | 6.4 |
| Attack paths | Incident and event data | Method not confirmed | 6.5 |
| Risk assessment | Risk-scoring engine | Fixed asset classification | 6.6 |
| Response and approval | Policy engine, authorisation | KL-4, KL-8 | 6.7 |
| Account restoration | Account status records | Cooldown rule undocumented | 6.8 |
| Simulated portal | Event submission | Synthetic nature | 6.9 |
| Audit logging | Audit records | KL-9; no screen supplied | — |

**Table 6.12 Summary of Modules, Dependencies and Related Issues**

# CHAPTER 7

# TESTING AND EVALUATION

## 7.1 INTRODUCTION

This chapter describes how the prototype is to be tested and which evidence is currently available. Only tests that are supported by evidence are reported as having been performed. At the time of writing, no recorded test run has been supplied for this report. The only evidence is the set of nine screenshots in Chapter 6, which show what the interface displayed during one demonstration session. Where a screenshot bears on a test case, the actual outcome records what it shows and the status is given as Observed, Partly observed, Inconclusive or Defect observed. These are not formal pass or fail results. All other test cases remain to be verified. This is a statement about the state of the evidence, not a statement that the tests have failed.

## 7.2 SOFTWARE FUNCTIONALITY TESTING AND MACHINE-LEARNING EVALUATION

Two distinct kinds of evaluation are needed and must not be confused. Software functionality testing asks whether the system behaves as designed: that the ML service returns a valid label, that the risk engine computes the expected score and that approval changes the state of an action. Machine-learning performance evaluation asks how well the classifier generalises: how accurate it is on independent labelled data, how errors are distributed among classes and how often it raises false alarms. Passing functionality tests says nothing about model accuracy, and a high model accuracy on synthetic data says nothing about the correctness of the workflow software.

## 7.3 TEST ENVIRONMENT

| Item | Detail |
|---|---|
| Front end | [browser and version] |
| Back end | [Node.js version, port] |
| ML service | [Python version, port] |
| Database | SQLite, [file location] |
| Test tools | [unit-test frameworks used, if any] |
| Test date and tester | [to be completed] |

**Table 7.1 Test Environment**

## 7.4 TEST CASES

Tables 7.2 to 7.5 give the planned test cases. The expected outcome states the behaviour that a correct implementation should show. The actual outcome and status must be filled in only from a recorded test run.

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-01 | ML service | Service responds | Service started | A prediction request is accepted | ML Engine indicator shows FALLBACK (Figure 6.1) | Inconclusive |
| TC-02 | ML service | Valid class output | A valid five-feature vector | One of the five category labels returned | Categories displayed (Figures 6.4, 6.5); ML engine in FALLBACK, so source is unconfirmed | Inconclusive |
| TC-03 | ML service | Confidence range | A valid five-feature vector | Confidence between 0 and 1 equal to the maximum class probability | Confidence shown as 88% (Figure 6.4); link to predict_proba not verified | Partly observed |
| TC-04 | ML service | Invalid input | Missing or non-numeric feature | Request rejected with an error | To be verified | To be verified |
| TC-05 | ML service | Repeatability | Same vector sent twice | Identical label and confidence | To be verified | To be verified |
| TC-06 | Risk engine | Score calculation | Known category, confidence and criticality | Score equals the value computed by hand from the documented rule | Score 100/100 displayed (Figure 6.6); formula not checked by hand | Partly observed |
| TC-07 | Risk engine | Repeated events | Same incident type repeated | Score does not decrease and increases as defined | To be verified | To be verified |
| TC-08 | Risk engine | Asset criticality | Incoming alert | Criticality is HIGH (current design) | Target asset shown as Production Endpoint (Tier 2) (Figure 6.6) | Partly observed |

**Table 7.2 Test Cases for the ML Service and Risk Engine**

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-09 | Policy engine | Rule selection | Known category and risk score | Action matches the documented rule | To be verified | To be verified |

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-10 | Policy engine | Approval required | Action designated as needing approval | Action held as pending | Approve and Reject controls shown (Figure 6.7) | Partly observed |
| TC-11 | Approval | Approve | Pending action | Status changes to approved and action proceeds | Badge APPROVED shown (Figure 6.7); approval action not captured | Partly observed |
| TC-12 | Approval | Reject | Pending action | Status changes to rejected; action not executed | To be verified | To be verified |
| TC-13 | Restoration | Restore account | Restricted account | Account returns to restored state | Restore Access control shown for BLOCKED account (Figure 6.8); restoration not captured | Partly observed |
| TC-14 | Audit log | Event recorded | Complete event-to-approval scenario | Every initial event, inference and policy decision appears | No audit-log screen in navigation (Section 6.11) | To be verified (KL-9) |
| TC-15 | Database | Persistence | Data stored, back end restarted | Data remains after restart | To be verified | To be verified (KL-6) |

**Table 7.3 Test Cases for the Policy Engine and Analyst Workflow**

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-16 | Dashboard | Display | Processed incidents exist | Incident information displayed correctly | Counters and telemetry displayed (Figure 6.1) | Observed |
| TC-17 | Dashboard | Fast-forward | Use fast-forward control | Backed by an existing API; no error | Control present (Figure 6.1); operation not exercised | To be verified (KL-5) |
| TC-18 | Threat detection | Manual inference | Known malicious-pattern vector | Result comes from the real ML service, not a stub | Event selector shown, no run captured (Figure 6.3) | To be verified (KL-3) |
| TC-19 | Response workflow | Recommendation fields | Incident with recommendation | Correct action text displayed | Recommended-action chip is blank (Figure 6.7) | Defect observed (KL-4) |
| TC-20 | Portal | Event generation | Simulated login failures | Events reach the analysis workflow | Events for demo_user listed (Figures 6.2, 6.9) | Observed |

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-21 | Integration | End-to-end scenario | Failed logins, then success, then sensitive access | Category, risk, policy decision and approval step shown | Single run captured across Figures 6.2 to 6.8; not repeated | Partly observed |

**Table 7.4 Test Cases for the User Interface and Integration**

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| TC-22 | Authorisation | Role spoofing | Request with a modified role header | Request refused unless identity and role are verified | To be verified | To be verified (KL-8) |
| TC-23 | Deployment | Container networking | Back end and ML service in Docker | Back end reaches the ML service by service name | To be verified | To be verified (KL-7) |
| TC-24 | Start-up | Data retention | Existing data and server restart | Existing records not erased | To be verified | To be verified (KL-6) |

**Table 7.5 Test Cases for Security and Deployment**

## 7.5 EXISTING AUTOMATED TESTS

A preliminary review confirmed only a small number of unit-test assertions in the project. This is not sufficient to establish the correctness of the system. The existing tests, their location and their results are to be recorded in Appendix B once the test suite has been run, and the test cases above are intended to extend coverage in a systematic way.

## 7.6 EVALUATION MEASURES FOR THE MACHINE-LEARNING MODEL

The Random Forest has not been evaluated on an independent labelled test set, so no accuracy or other performance figure is reported. The measures in Table 7.6 are those required for a credible evaluation.

| Measure | Definition | Why it matters |
|---|---|---|
| Accuracy | Correct predictions divided by all predictions | Overall rate; misleading with unequal classes |
| Precision (per class) | TP / (TP + FP) | Cost of false alarms for each category |
| Recall (per class) | TP / (TP + FN) | Share of true events of each category detected |
| F1-score | Harmonic mean of precision and recall | Balances the two |
| Confusion matrix | Counts of true versus predicted class | Shows which categories are confused |

| Measure | Definition | Why it matters |
|---|---|---|
| False-positive rate | FP / (FP + TN) | Alert burden on analysts |
| Inference latency | Time to return a prediction | Suitability for interactive use |
| Response-workflow reliability | Share of workflow runs that complete correctly | Measures the software workflow, not the model |

**Table 7.6 Evaluation Measures Required for the Classifier**

A valid evaluation requires independent training, validation and test splits, evaluation on data not generated by the same rules as the training data, and per-class reporting, particularly for rarer categories. Evaluation on suitable public cybersecurity datasets is proposed as future work. Calibration of the confidence value should also be assessed before it is interpreted as a probability.

# CHAPTER 8

# RESULTS AND DISCUSSION

## 8.1 BASIS OF THE RESULTS

Results in this chapter are limited to what is supported by the supplied evidence. The evidence consists of nine screenshots of the running prototype captured on 10 October 2026 (Figures 6.1 to 6.9). No server logs, recorded test runs or model-evaluation records have been supplied. The chapter therefore reports what the screenshots show, states what they do not show, and does not report numerical performance results. The values that appear in the screenshots, such as a risk score of 100 and a confidence of 88%, are single observations from one demonstration session and are not measurements of performance.

## 8.2 WHAT THE PROTOTYPE IS DESIGNED TO DEMONSTRATE

- An event sequence can be summarised by five features and classified into one of five categories by a Random Forest served from a separate FastAPI service.
- A deterministic engine can convert the category, model confidence and context into a risk score, and a rule-based policy engine can use it to select or control a response action.
- Selected response actions can be held for analyst approval or rejection, and affected accounts can be restored.
- The resulting information can be recorded and displayed through a dashboard, audit records and a simulated enterprise portal.

These are statements about the design of the prototype. The following sections compare them with what the screenshots show.

## 8.3 BEHAVIOUR OBSERVED IN THE SCREENSHOTS

| No. | Observation | Evidence | Interpretation |
|---|---|---|---|
| 1 | The interface provides eight analyst screens and a simulated portal | Figure 6.1 navigation; Figure 6.9 | The module structure of Chapter 6 exists as an interface |
| 2 | Dashboard shows 10 events, 1 detected threat and 1 active incident | Figure 6.1 | Counters are populated for the session |
| 3 | Engine health shows ML Engine as FALLBACK; Predictive Engine, Risk Engine ACTIVE; Database CONNECTED | Figure 6.1 | Source of classification output is unconfirmed (Section 8.5) |

| No. | Observation | Evidence | Interpretation |
|---|---|---|---|
| 4 | Events for demo_user from 192.168.1.55 are listed (FILE_ACCESS, SENSITIVE_FILE_ACCESS) | Figure 6.2 | The portal activity reaches the event store |
| 5 | Incident ALT-8329 is listed with stage, predicted stage, risk HIGH (100), confidence 88%, status OPEN | Figure 6.4 | A threat record is created from the events |
| 6 | Attack-path view shows observed behaviours and predicted next moves | Figure 6.5 | Display exists; method not confirmed |
| 7 | Risk report shows 100/100, CRITICAL and four calculation factors | Figure 6.6 | Score is displayed; formula not checked |
| 8 | Response screen shows APPROVED badge, Approve & Reject controls and a blank recommended-action chip | Figure 6.7 | Display defect consistent with KL-4 |
| 9 | demo_user is shown as BLOCKED with a Restore Access button | Figure 6.8 | A restriction and manual restore control exist |
| 10 | No audit-log screen is present in the navigation | Figure 6.1 | Audit visibility is not demonstrated (KL-9) |

**Table 8.1 Behaviour Observed in the Supplied Screenshots**

## 8.4 SEQUENCE OF EVENTS IN THE CAPTURED SESSION

Table 8.2 orders the timestamps visible in the screenshots. The ordering shows that the screens come from one session, but it does not show how the listed events relate to the incident, because the incident timestamp precedes the earliest event visible in Figure 6.2 and the event list is cut off. The timestamps are not used to calculate any response time.

| Time | Observation | Figure |
|---|---|---|
| 9:21:08 pm | Incident ALT-8329 created; status OPEN | Figure 6.4 |
| 9:21:09 pm | FILE_ACCESS event by demo_user, status SUCCESS | Figure 6.2 |
| 9:22:15 pm | FILE_ACCESS event | Figure 6.2 |
| 9:22:17 pm | SENSITIVE_FILE_ACCESS event | Figure 6.2 |
| 9:22:24 pm | SENSITIVE_FILE_ACCESS event | Figure 6.2 |
| 9:22:25 pm | FILE_ACCESS event | Figure 6.2 |

| Time | Observation | Figure |
|---|---|---|
| 9:22:27 pm | FILE_ACCESS event | Figure 6.2 |
| 9:27:03 pm | demo_user shown as BLOCKED | Figure 6.8 |

**Table 8.2 Timestamps Visible in the Captured Session (10/10/2026)**

## 8.5 IMPORTANT OBSERVATION: ML ENGINE IN FALLBACK STATE

The most important observation for the interpretation of the results is that the Engine Health panel reported the ML Engine as FALLBACK (Figure 6.1). The categories, confidence value and incident record seen in Figures 6.4 to 6.7 may therefore have been produced by a fallback mechanism rather than by the Random Forest. This can happen, for example, if the ML service is not reachable. The cause and the nature of the fallback are not established by the supplied material.

Consequently, the screenshots demonstrate the surrounding workflow – events, incident, risk display, recommendation, restriction and restore control – but they do not demonstrate the behaviour of the Random Forest classifier. To obtain evidence about the classifier, the experiment must be repeated with the ML service running and the ML Engine indicator showing a normal state, and the response of the ML service must be recorded.

## 8.6 WHAT CANNOT YET BE CONCLUDED

Because no independent evaluation has been carried out, the following conclusions are not supported and are not made in this report:

- that the Random Forest model classifies real or synthetic events accurately or reliably;
- that Orbit AI improves detection accuracy compared with other approaches;
- that Orbit AI reduces the time needed to respond to incidents;
- that the confidence value is a calibrated probability of an attack;
- that the interface labels such as Agentic Reasoning indicate generative-AI reasoning;
- that the system is secure, fully validated or suitable for enterprise deployment.

Even if the model produced very high accuracy on data from the same synthetic generator that created its training set, this would not support any of the conclusions above [11].

## 8.7 EXPERIMENTS REQUIRED

| No. | Experiment | Purpose | Priority |
|---|---|---|---|
| E1 | Repeat the demonstration with the ML service running and healthy; record the service response | Obtain evidence of Random Forest output | High |
| E2 | Hold-out evaluation on independently generated labelled data | First estimate of generalisation | High |

| No. | Experiment | Purpose | Priority |
|---|---|---|---|
| E3 | Evaluation on a suitable public cybersecurity dataset | Assess behaviour outside the synthetic generator | High |
| E4 | Per-class precision, recall and F1-score with confusion matrix | Identify confused categories | High |
| E5 | Confidence calibration check | Test whether confidence can be read as a probability | Medium |
| E6 | Inference-latency measurement | Check suitability for interactive use | Medium |
| E7 | Repeated end-to-end workflow runs including approval, rejection and restore | Measure workflow reliability | High |
| E8 | Policy-rule unit tests | Check that every rule gives the intended action | High |

**Table 8.3 Experiments Required to Obtain Supportable Results**

## 8.8 TEMPLATES FOR RECORDING MODEL RESULTS

Tables 8.4 and 8.5 are blank templates for recording model results once experiments E2 to E4 have been carried out. No values have been entered, and none should be entered unless they come from a recorded experiment.

| True \ Predicted | Monitoring | Auth. Anomaly | Account Comp. | Data Coll. | Priv. Esc. |
|---|---|---|---|---|---|
| Monitoring | — | — | — | — | — |
| Authentication Anomaly | — | — | — | — | — |
| Account Compromise | — | — | — | — | — |
| Data Collection | — | — | — | — | — |
| Privilege Escalation | — | — | — | — | — |

**Table 8.4 Confusion Matrix Template (Rows: True Class; Columns: Predicted Class)**

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| Monitoring | — | — | — | — |
| Authentication Anomaly | — | — | — | — |
| Account Compromise | — | — | — | — |
| Data Collection | — | — | — | — |
| Privilege Escalation | — | — | — | — |

**Table 8.5 Per-Class Performance Template**

## 8.9 DISCUSSION

### 8.9.1 What the screenshots show about the workflow

Taken together, the screenshots show a coherent demonstration path: portal activity by demo_user becomes events, an incident is created with a stage, a predicted stage, a risk score and a confidence value, a response is recommended with approval controls, and an account restriction with a manual restore control is available. This supports the claim that the surrounding workflow has been implemented as a prototype interface. It does not support claims about the quality of the classification, because of the fallback state in Section 8.5.

### 8.9.2 Role of analyst approval

Holding selected actions for approval limits the consequences of an incorrect classification, which matters given the cost of errors highlighted in the literature [11]. The prototype also shows the associated trade-off: each approval adds a human step and therefore delay. The balance between efficiency and oversight can only be assessed with measurements of response time and error rates, which have not been made.

### 8.9.3 Interface wording and the nature of the automation

Some interface labels, such as Agentic Response Workflow, Agent Engine Active and Agentic Reasoning, suggest autonomous agent behaviour. The design described in this report is deterministic rule-based automation combined with a conventional classifier. Unless the source code shows otherwise, the labels should be understood as names for the interface sections, and the project should not be described as using a large-language-model agent.

### 8.9.4 Impact of the identified issues

The issues listed in Chapter 9 influence how any observation should be interpreted. For example, the blank recommended-action chip means that a screenshot of the response screen cannot be used to show which action was recommended, and the inconsistency between HIGH (100) and CRITICAL means that risk labels must be checked against the scoring rule before they are reported.

## 8.10 THREATS TO VALIDITY

- **Construct validity.** Synthetic classes are defined by the developers, so correct classification of them does not show that real attacks are recognised.
- **Internal validity.** Unresolved integration defects and the ML fallback state could alter what is observed on screen.
- **External validity.** The prototype operates on simulated events from one user in a single demonstration session.
- **Conclusion validity.** No repeated runs or statistical analysis have been performed, so no performance claim can be made.

# CHAPTER 9

# LIMITATIONS AND FUTURE ENHANCEMENTS

## 9.1 INTRODUCTION

This chapter records the limitations of the prototype and proposes future work. The first ten limitations were identified in a preliminary code review. They have not yet been re-checked against the latest source code and are therefore treated as open until each one has been confirmed as resolved, although the screenshots provide some evidence for some of them. Further issues observed in the screenshots are listed separately. Future enhancements are proposals and are not completed features.

## 9.2 LIMITATIONS IDENTIFIED IN PRELIMINARY REVIEW

| ID | Limitation | Implication | Proposed Remediation (Future Work) | Status |
|---|---|---|---|---|
| KL-1 | Training data is synthetic, not a demonstrated real-world benchmark | Model may not generalise; results may reflect the generator | Train and test on suitable public or real datasets | Open |
| KL-2 | No verified accuracy, precision, recall, F1 or benchmark results | Detection quality is unknown | Independent evaluation (Chapter 7) | Open |
| KL-3 | Manual inference interface may call a stub and show a benign result | Misleading output to the analyst | Connect the interface to the ML service and test | Re-verify (Figure 6.3 shows no run) |
| KL-4 | Response workflow may read incorrect recommendation properties | Wrong or missing action information | Align front-end fields with the API and add tests | Evidence: blank action chip (Figure 6.7) |
| KL-5 | Dashboard fast-forward may depend on a non-existent API | Feature fails or is not functional | Implement the API or remove the control | Re-verify (button present, Figure 6.1) |
| KL-6 | Back end may use sequelize.sync with force enabled | Database contents erased at start-up | Remove force; use controlled migrations | Re-verify |
| KL-7 | Docker back end may use localhost for the ML service | Containerised back end cannot reach ML service | Use the service hostname through configuration | Re-verify |
| KL-8 | User role taken from a request header | Role can be forged | Verified authentication and role-based access control | Re-verify |
| KL-9 | Audit log may omit initial events, inferences and policy decisions | Incomplete traceability | Log every stage of the workflow | Re-verify (no audit screen supplied) |

| ID | Limitation | Implication | Proposed Remediation (Future Work) | Status |
|---|---|---|---|---|
| KL-10 | Only a small number of unit-test assertions confirmed | Low confidence in correctness | Extend unit and integration tests | Open |

**Table 9.1 Known Limitations, Implications and Proposed Remediation**

## 9.3 ADDITIONAL ISSUES OBSERVED IN THE SCREENSHOTS

| ID | Observation | Implication | Proposed Action |
|---|---|---|---|
| OI-1 | ML Engine health shows FALLBACK (Figure 6.1) | Displayed classifications may not come from the Random Forest | Confirm the cause; repeat the demonstration with a healthy ML service |
| OI-2 | Recommended Action chip is blank (Figure 6.7) | The recommended action is not visible to the analyst | Correct the displayed property (see KL-4) |
| OI-3 | Incident risk shown as HIGH (100) but CRITICAL on the risk report (Figures 6.4, 6.6) | Inconsistent risk labelling | Define one mapping from score to label |
| OI-4 | Status APPROVED shown while Approve and Reject controls remain visible (Figure 6.7) | Unclear state of the approval workflow | Disable or hide controls after a decision |
| OI-5 | Interface labels use Agentic terms (Figure 6.7) | May be misread as generative-AI agent behaviour | Rename or document the labels |
| OI-6 | Target resource is N/A for visible events (Figure 6.2) | Event context is incomplete | Populate the field or remove the column |

**Table 9.2 Additional Issues Observed in the Supplied Screenshots**

## 9.4 DISCUSSION OF THE LIMITATIONS

### 9.4.1 Data and model validity

The model is trained on synthetic data and has no demonstrated performance on real events (KL-1, KL-2). Research on machine learning for intrusion detection stresses that evaluation is difficult and that the costs of errors are high [11]. Until independent evaluation has been done, the classification output should be treated as an illustration of the workflow and not as dependable detection. The confidence value is a model score and has not been shown to be calibrated. The fallback state of the ML engine (OI-1) adds a further uncertainty about the origin of the displayed output.

### 9.4.2 Front end and back end integration

The reported stub in manual inference (KL-3), the blank recommendation chip (KL-4, OI-2) and the dependency of the fast-forward operation on a missing API (KL-5) are integration defects. The inconsistent risk labels (OI-3) and the unclear approval state (OI-4) are interface consistency

defects. They affect the credibility of what the analyst sees and could produce misleading displays. They should be fixed and covered by integration tests.

### 9.4.3 Data persistence and deployment

If synchronisation with the force option is enabled (KL-6), the database is reset at start-up, which defeats the purpose of audit records and incident history. Use of localhost for the ML service in a container (KL-7) would prevent the back end from reaching the model service in a Docker deployment, and could itself be a cause of the ML fallback state if the same configuration is used. SQLite is suitable for a demonstration but not for multi-user operational use.

### 9.4.4 Security

Using a role header as the basis for authorisation (KL-8) allows any client to claim any role. Because approval and restoration are privileged actions, this undermines the human-oversight design. Robust authentication, session management and server-side role-based access control are required.

### 9.4.5 Audit coverage and testing

Audit records that do not capture every initial event, inference and policy decision (KL-9) limit post-incident review, and the absence of an audit screen in the supplied interface means that audit visibility is not demonstrated. A small number of unit-test assertions (KL-10) cannot support confidence in correctness.

### 9.4.6 Design limitations

- Asset classification appears fixed (Production Endpoint, Tier 2, in Figure 6.6, and HIGH criticality for incoming alerts), so the risk score does not reflect asset importance.
- The model uses five aggregate count features, which cannot represent timing, order or context.
- The policy engine is a fixed rule set and does not adapt.
- A Google GenAI dependency may be present, but generative-AI functionality is not claimed.
- The prototype uses simulated events and is not connected to real security-event sources.

## 9.5 REQUIREMENTS FOR REAL-WORLD DEPLOYMENT

The prototype would need considerable further work before real deployment, including independent model validation, strong authentication and authorisation, secure configuration and secret management, a production-grade database with backup and migration, complete audit logging, operational monitoring, secure deployment of the services and policy testing with change control. Compliance with the organisation’s own procedures and relevant guidance, such as that for IDPS deployment [8], would also have to be established.

## 9.6 FUTURE ENHANCEMENTS

The following are proposals for future work and are not features of the current prototype.

| Enhancement | Description | Addresses |
|---|---|---|
| Public-dataset evaluation | Evaluate on suitable public cybersecurity datasets with independent train, validation and test splits | KL-1, KL-2 |
| Per-class analysis | Report precision, recall, F1-score and confusion matrix for each category | KL-2 |
| Calibration | Assess and, if needed, calibrate model confidence | Section 5.7 |
| Explainability | Add explanation methods, for example SHAP, if implemented in the future [7] | Section 2.8 |
| Dynamic asset criticality | Replace the fixed asset classification with asset-based criticality | Section 5.8 |
| Role-based access control | Verified authentication and server-side authorisation | KL-8 |
| Complete audit logging | Record every event, inference and policy decision and provide an audit screen | KL-9 |
| Interface consistency | Fix blank fields, risk labels and approval state display | OI-2 to OI-4 |
| Policy testing and safeguards | Test rules and add approval safeguards for rule changes | Section 5.9 |
| ATT&CK mapping | Map categories to ATT&CK tactics and techniques [3] | Section 2.4 |
| Real event sources | Integrate real security-event sources after validation | Scope |
| Integration tests | Automated tests for the full workflow | KL-10 |

**Table 9.3 Proposed Future Enhancements (Not Implemented)**

## 9.7 ETHICAL AND RESPONSIBLE-USE CONSIDERATIONS

Orbit AI processes only simulated events and simulated accounts. If it were applied to real systems, the logs it would use would contain personal and organisational data, so lawful collection, access control and retention would have to be addressed. A wrong classification could restrict a legitimate user, which is why human approval for selected actions and a clear restoration path are part of the design. Decision logic and its limits should be openly documented so that analysts do not over-trust the system.

# CHAPTER 10

# CONCLUSION

This project addressed the difficulty of interpreting sequences of security events and responding to them in a controlled way. Orbit AI is a cybersecurity demonstration prototype that summarises a sequence of events into five features, classifies the pattern into one of five categories using a Random Forest served by a FastAPI service, calculates a deterministic risk score, applies rule-based policy to select or control a response action and holds selected actions for analyst approval. A React dashboard, a Node.js and Express back end with an SQLite database, audit records and a simulated enterprise portal make the workflow observable.

Machine learning has a deliberately narrow role in the system: it assigns a category and a confidence value, and the confidence is a model score and not a calibrated probability of attack. Risk scoring and policy evaluation are deterministic and inspectable, and the policy engine is rule-based logic and not an autonomous reasoning agent. Analyst approval for selected actions keeps a human decision in the path of consequential responses, consistent with the concern in the security-orchestration literature that automation must be carefully controlled [9], [10].

The report has been explicit about the limits of the work. The training data is synthetic, no accuracy or other performance measures have been established, asset criticality is fixed at HIGH, and several implementation issues from a preliminary review need to be re-verified and resolved. For these reasons the prototype is not production-ready and makes no claim of reliable real-world threat detection. The future work described in Chapter 9, beginning with independent evaluation on suitable datasets, stronger authentication and authorisation, and complete audit logging, indicates the direction in which the prototype would have to be developed before any operational use.

## REFERENCES


[1] P. García-Teodoro, J. Díaz-Verdejo, G. Maciá-Fernández, and E. Vázquez, “Anomaly-based network intrusion detection: Techniques, systems and challenges,” *Computers & Security*, vol. 28, no. 1-2, pp. 18-28, 2009, doi: 10.1016/j.cose.2008.08.003.

[2] M. Husák, J. Komárková, E. Bou-Harb, and P. Čeleda, “Survey of attack projection, prediction, and forecasting in cyber security,” *IEEE Communications Surveys & Tutorials*, vol. 21, no. 1, pp. 641-660, 2019, doi: 10.1109/COMST.2018.2871866.

[3] B. E. Strom *et al.*, “MITRE ATT&CK: Design and philosophy,” The MITRE Corporation, Bedford, MA, Tech. Rep. MTR180153, 2020. [Online]. Available: https://www.mitre.org/news-insights/publication/mitre-attck-design-and-philosophy

[4] M. Landauer, F. Skopik, M. Wurzenberger, and A. Rauber, “System log clustering approaches for cyber security applications: A survey,” *Computers & Security*, vol. 92, Art. no. 101739, 2020, doi: 10.1016/j.cose.2020.101739.

[5] C. Yin, Y. Zhu, J. Fei, and X. He, “A deep learning approach for intrusion detection using recurrent neural networks,” *IEEE Access*, vol. 5, pp. 21954-21961, 2017, doi: 10.1109/ACCESS.2017.2762418.

[6] S. Noel and S. Jajodia, “Managing attack graph complexity through visual hierarchical aggregation,” in *Proc. 2004 ACM Workshop on Visualization and Data Mining for Computer Security (CCSC)*, 2004, pp. 109-118, doi: 10.1145/1029208.1029225.

[7] F. Charmet *et al.*, “Explainable artificial intelligence for cybersecurity: a literature survey,” *Annals of Telecommunications*, vol. 77, no. 11-12, pp. 789-812, 2022, doi: 10.1007/s12243-022-00926-7.

[8] K. Scarfone and P. Mell, “Guide to intrusion detection and prevention systems (IDPS),” NIST Spec. Publ. 800-94, Feb. 2007, doi: 10.6028/NIST.SP.800-94.

[9] C. Islam, M. A. Babar, and S. Nepal, “A multi-vocal review of security orchestration,” *ACM Computing Surveys*, vol. 52, no. 2, Art. no. 37, pp. 1-45, Apr. 2019, doi: 10.1145/3305268.

[10] C. Islam, M. A. Babar, and S. Nepal, “Design and generation of a set of declarative APIs for security orchestration,” *IEEE Transactions on Services Computing*, vol. 17, no. 1, pp. 127-141, Jan.–Feb. 2024, doi: 10.1109/TSC.2023.3336666.

[11] R. Sommer and V. Paxson, “Outside the closed world: On using machine learning for network intrusion detection,” in *Proc. 2010 IEEE Symp. Security and Privacy (S&P)*, 2010, pp. 305-316, doi: 10.1109/SP.2010.25.

[12] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825-2830, 2011. [Online]. Available: https://jmlr.org/papers/v12/pedregosa11a.html

# APPENDIX

## APPENDIX A: PROJECT TEAM AND GUIDE DETAILS

| No. | Name | Register number | Email |
|---|---|---|---|
| 1 | A. Mohamed Ismail | 95192309027 | 23ad027@psr.edu.in |
| 2 | K. Siva Subramanian | 95192309048 | 23ad048@psr.edu.in |
| 3 | M. Kavin Kumar | 95192309016 | 23ad016@psr.edu.in |
| 4 | P. Ram Kishore | 95192309042 | 23ad042@psr.edu.in |

**Table A.1 Project Team**

| Item | Detail |
|---|---|
| Name | Mr. D. Jeba Kingsley |
| Designation | Assistant Professor |
| Department | Artificial Intelligence and Data Science |
| Email | jebakingsley@psr.edu.in |
| Institution | P.S.R. Engineering College, Sivakasi, Tamil Nadu, India |

**Table A.2 Project Guide**

## APPENDIX B: SOURCE-CODE VERIFICATION CHECKLIST

The following items must be completed from the repository before final submission. No source files or credentials are reproduced here.

| Item | Information to record | Status |
|---|---|---|
| API endpoints | List of back-end and ML-service endpoints, methods and payloads | To be completed |
| Database models | Tables, fields and relationships | To be completed |
| Training pipeline | Class-generation rules for the synthetic data | To be completed |
| Existing unit tests | Files, number of assertions and results | To be completed |
| Known limitations | Re-check KL-1 to KL-10 against the latest commit | To be completed |
| GenAI usage | Confirm whether the Google GenAI package is called anywhere | To be completed |
| Runtime versions | Node.js, Python and package versions | To be completed |

**Table B.1 Items to Record from the Source Code**

## APPENDIX C: ADDITIONAL FIGURES TO BE INSERTED

**[PLACEHOLDER – NOT A REAL SCREENSHOT]**

*Required: Entity-relationship diagram produced from the Sequelize models once the schema has been verified.*

**Figure A.1 Entity-Relationship Diagram of the Orbit AI Database (Screenshot to Be Inserted)**

**[PLACEHOLDER – NOT A REAL SCREENSHOT]**

*Required: Deployment diagram based on the supplied Docker configuration, if deployment configuration is provided.*

**Figure A.2 Deployment Architecture of the Orbit AI Prototype (Screenshot to Be Inserted)**

## APPENDIX D: RISK-SCORING AND POLICY SUMMARY TEMPLATES

| Factor | Value or weight | Source file |
|---|---|---|
| Base severity per category | [to be completed] | [file] |
| Model confidence contribution | [to be completed] | [file] |
| Asset criticality (HIGH) | [to be completed] | [file] |
| Repeated-event increment | [to be completed] | [file] |

**Table D.1 Risk-Scoring Parameters (to be Completed from Source)**

| No. | Condition | Action | Approval needed |
|---|---|---|---|
| 1 | [to be completed] | [to be completed] | [yes/no] |
| 2 | [to be completed] | [to be completed] | [yes/no] |
| 3 | [to be completed] | [to be completed] | [yes/no] |

**Table D.2 Policy Rules (to be Completed from Source)**

## APPENDIX E: TEST-CASE TEMPLATE

| ID | Module | Objective | Input / precondition | Expected outcome | Actual outcome | Status |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |

**Table E.1 Blank Test-Case Record**

## APPENDIX G: PUBLICATION AND ORIGINALITY EVIDENCE

### G.1 Conference Submission Acknowledgements

Table G.1 records the three conference submissions of the Orbit AI paper, as shown in the acknowledgement e-mails from Microsoft CMT (Figures G.1 to G.3). Each e-mail states that the submission has been created. The e-mails do not state that a paper has been accepted, reviewed or published, and none of these outcomes is claimed in this report.

| Track | Conference | Paper ID | E-mail Time | Status per E-mail |
|---|---|---|---|---|
| CYPHER2026 | International Conference on Cybersecurity, Privacy, and Human-centric Emerging Research | 255 | 8:39 PM | Submission created |
| SPECTRUM2027 | 8th International Conference on Current Advancement in Science & Technology (Theme: Advancements in Modern Physics: Use of AI, Machine Learning and Emerging Applications) | 5 | 8:48 PM | Submission created |
| HACER2026 | 2nd International Conference on Harnessing Artificial Intelligence for Cybersecurity, Healthcare & Climate Resilience | 202 | 9:00 PM | Submission created |

**Table G.1 Conference Submission Records**




**Figure G.1 CMT Acknowledgement for Paper ID 255 (CYPHER2026)**

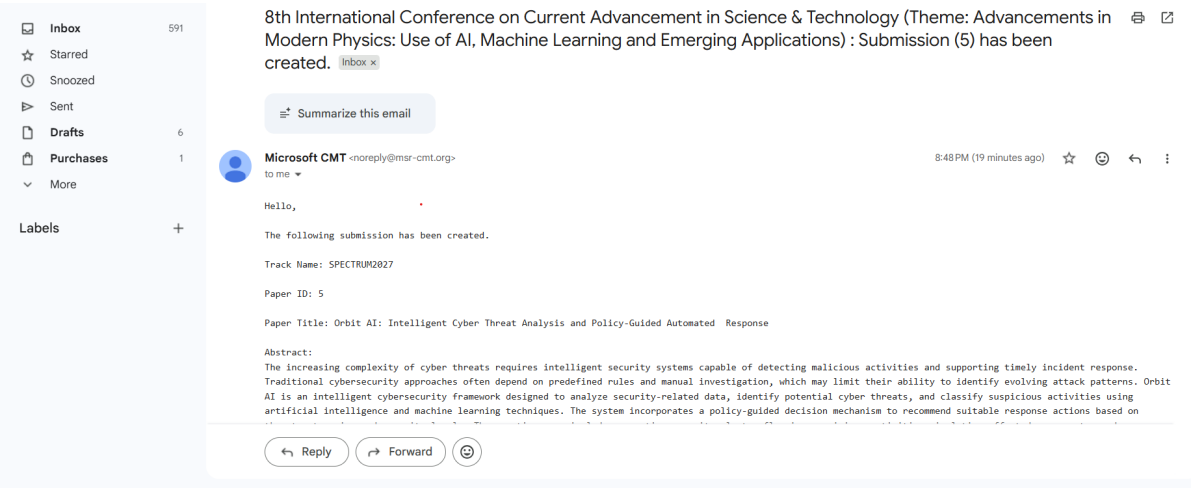



**Figure G.2 CMT Acknowledgement for Paper ID 5 (SPECTRUM2027)**

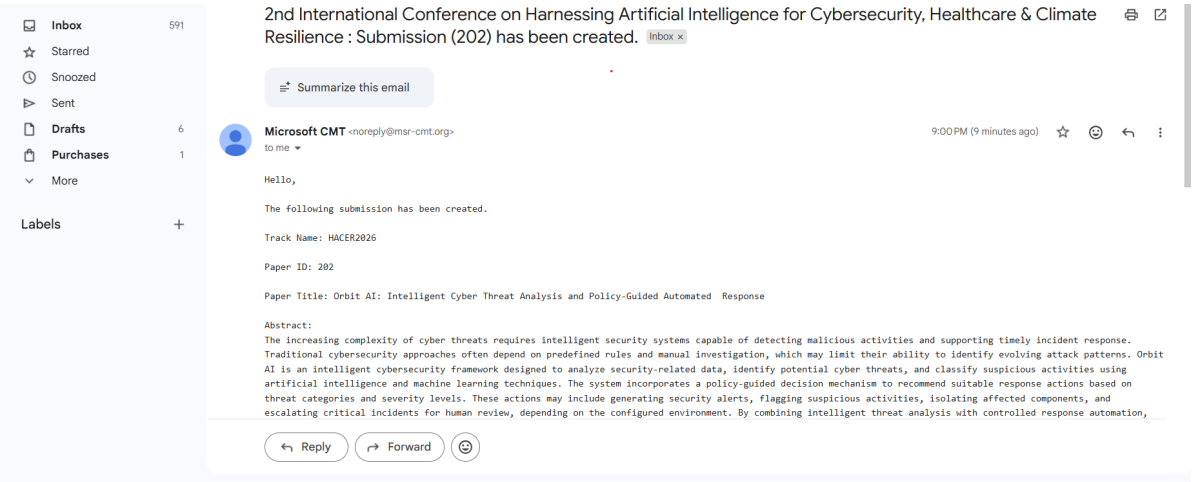



**Figure G.3 CMT Acknowledgement for Paper ID 202 (HACER2026)**

## G.2 Similarity Report for the Conference Paper

Figure G.4 and Table G.2 reproduce the overview page of the Turnitin similarity report (submission ID trn:oid:::79569:927122307) for the file SUB-ID-787563.pdf, submitted on 10 October 2026. This report applies to the seven-page conference paper (5,414 words) and not to this project report. A separate similarity check of this report is to be attached when it becomes available. The bibliography was excluded from the Turnitin comparison. A similarity percentage describes overlap with indexed sources; it is not by itself a judgement of originality.

| Item | Value |
|---|---|
| Overall similarity | 4% |
| Not cited or quoted | 15 matches (3%) |
| Missing quotations | 3 matches (1%) |
| Missing citation | 0 matches (0%) |
| Cited and quoted | 0 matches (0%) |

| Item | Value |
|---|---|
| Source types | Internet sources 3%; publications 1%; student papers 2% |
| Integrity flags | 0 |
| Submission date | 10 October 2026, 01:44 AM |
| Document size | 7 pages; 5,414 words; 31,141 characters |

**Table G.2 Turnitin Similarity Summary for the Conference Paper**

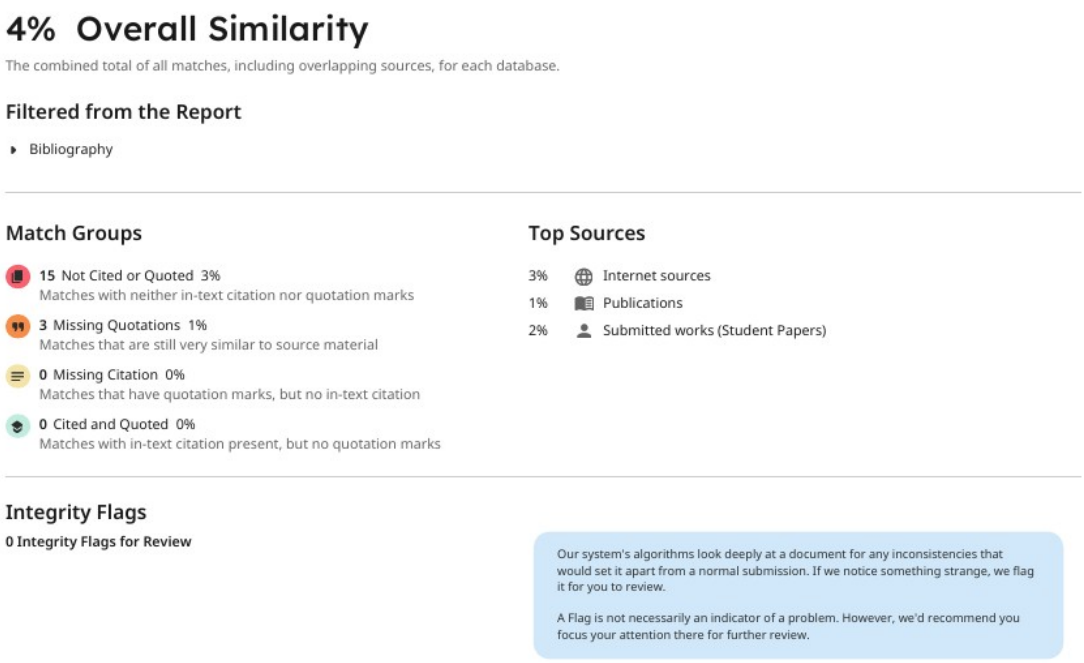



**Figure G.4 Turnitin Integrity Overview for the Conference Paper**

The technical description in the conference paper differs in some respects from the description in this report (for example, in the database technology and in the method used for predicting attack stages). Before final submission the team should reconcile the two documents against the source code, so that both describe the same implementation.